

SoK: Offline Finding Protocols for Lightweight Location Tracking

Akshaya Kumar
Georgia Institute of Technology
Atlanta, Georgia, USA
akshayakumar@gatech.edu

Carolina Ortega Perez
Cornell University
New York, New York, USA
carolina@cs.cornell.edu

Joseph Jaeger
Georgia Institute of Technology
Atlanta, Georgia, USA
josephjaeger@gatech.edu

Thomas Ristenpart
University of Toronto
Toronto, Ontario, Canada
ristenpart@cs.toronto.edu

Michael A. Specter
Georgia Institute of Technology
Atlanta, Georgia, USA
specter@gatech.edu

ABSTRACT

Offline finding (OF) protocols—such as Apple’s Find My, Google’s Find Hub, Samsung’s SmartThingsFind, and Tile—enable hundreds of millions of users to track their belongings via Bluetooth-based tracker tags. However, their scale and tracking capabilities give rise to privacy risks for tag owners and bystanders, as well as safety risks for victims of tag-facilitated stalking. In response, academics and practitioners have suggested cryptographic and non-cryptographic mitigations to improve privacy and anti-stalking protections, working to navigate complex and subtle tensions between these goals. The result is a large landscape of privacy goals, threat models, protocol designs, implementations, and analyses.

In this work, we systematize the OF protocol landscape. We gather and analyze a corpus of 49 research papers and OF protocol technical specifications, and use it to develop a taxonomy capturing the functionality, security, and privacy goals of OF protocols. We use the taxonomy to guide a focused assessment of the four major OF deployments along with six academic constructions, comparing design choices, consolidating known attacks, and analyzing the designs’ trade-offs between privacy, security, abusability, and efficiency. We provide a simple OF protocol that achieves most security goals, and which clarifies the essential cryptographic components underlying OF protocols. We also provide a survey of physical layer attacks and usability issues that undermine protections in practice. Finally, we discuss open problems and potential research directions towards secure, interoperable, and abuse-resistant OF systems.

1 INTRODUCTION

Over the past decade, Bluetooth-based offline finding (OF) systems have transformed how users track their belongings. First popularized by Tile in 2014 [60], OF systems leverage large crowd-sourced networks of participating “finder” devices to physically locate tracker tags. Tags can be as small as a coin, and use Bluetooth to advertise themselves to finder devices, which forward the advertisement together with the finder’s current location to the vendor’s backend. The tag’s owner can then access this tracking information. OF systems now underpin a massive, always-on location-tracking infrastructure supporting hundreds of millions of trackers via networks operated by Apple [3], Google [30], Samsung [62], and Life360 (which bought Tile) [50].

The very properties that make OF systems effective for finding items also introduce a variety of security, privacy, and safety risks. In terms of privacy, tags could be used by OF network operators

to track tag owners or finders [44, 45, 52, 83]. At the same time, tags are a quintessential dual-use technology: malicious tag owners can slip them into a victim’s bags, clothing, or vehicle to easily track their location. Trackers are routinely implicated in stalking, theft, physical assault, murder, and other criminal activities [8, 15, 17, 28, 55, 71, 72]. In response, vendors introduced “anti-stalking” features that notify users when an unknown tracker is persistently detected nearby [5, 6, 23, 48, 51, 63, 70]. Complications arise here given tensions between threat models: naive anti-stalking features sacrifice owner privacy (and vice versa).

These challenges are compounded by the proprietary and undocumented nature of OF protocols requiring the academic community to reverse engineer them individually [13, 39, 46, 53, 61, 82]. Unsurprisingly, these papers and others [10, 16, 49, 52, 54, 57, 83, 84] found attacks that circumvent privacy and anti-abuse mitigations. Simultaneously, researchers formalized various cryptographic properties for OF systems and proposed secure protocol constructions [19, 20, 22, 25, 37, 56]. The result is a patchwork and complex landscape, with so far no comprehensive treatment of threat models and security, privacy, and safety goals, nor a clear explanation of the core mechanisms that underlying OF protocols use to achieve them.

In this work, we provide the first comprehensive systematization of security, privacy, and anti-stalking in the context of OF protocols. We review 40 academic papers, including protocol designs, attacks, measurement, and user studies, as well as nine industry specifications. To start, we use the various papers and available specifications to reconstruct and compare the protocol designs from the major tag vendors (i.e., Apple, Google, Samsung, and Life360). Based on this, we describe the general OF protocol phases and detail a relatively simple protocol that can be viewed as a “textbook” version of how OF protocols can be built.

We then define 11 security properties for OF protocols, spanning privacy guarantees for all protocol participants as well as protections against abuse. Ten of these properties are derived from descriptions in our corpus, while one is a new goal we propose after identifying unaddressed gaps. We partition the 11 goals into *baseline* goals, which are specifically targeted and achieved by most vendors and can be thought of as the core security and privacy goals of an OF system, and *additional* goals, which are more advanced goals not targeted or achieved by most protocols but ones that would improve privacy and abuse resistance.

We evaluate both deployed protocols and academic proposals against our taxonomy. We find that no system achieves all baseline goals, let alone the additional ones—an unsurprising result given the

inherent tensions between many security objectives. We provide intuition for the design decisions and vulnerabilities that prevent protocols from achieving specific properties, and illustrate the trade-offs that arise when balancing privacy and abuse prevention.

Based on our findings, we identify promising directions for future research and policy to guide the development of usable, privacy-preserving OF systems that minimize abuse while respecting the practical constraints of large-scale deployment.

Summary. We provide the first systematization of the security, privacy, and safety landscape of OF protocols. In this work, we:

- contribute a corpus of relevant papers between 2014 and 2025, which we will submit as an artifact;
- contribute a comprehensive set of security, privacy, and anti-abusability goals and corresponding threat models to evaluate the security of OF protocols;
- describe the OF protocol implementations of Apple, Google, Samsung, and Tile, identifying differences in cryptographic design and evaluating them relative the security goals;
- discuss open research and policy challenges in balancing efficiency, privacy, and abuse prevention.

2 BACKGROUND

We begin by reviewing the origins and security analyses of OF systems to motivate the need for a systematization. We then provide background on Bluetooth Low Energy (BLE), the core communication layer underlying OF networks.

The evolution of OF networks. The idea of crowdsourcing to track objects using Bluetooth-capable tags and location-enabled smartphones was proposed in early location-tracking systems [24]. In the 2010s, this vision was commercialized by companies such as TrackR [29], Tile [50], Nut [59], and Cube [74]. However, these systems offered little in the way of privacy or anonymity and finder networks were limited to application users.

Apple's introduction of AirTags and the Find My network [41] marked a turning point, promising strong cryptographic guarantees for the security and privacy of users. Samsung and Google soon launched their own finder networks [31, 64]. Unlike earlier systems with small user bases, these major vendors could instantly bootstrap massive finder networks from their global smartphone ecosystems.

Despite their scale, these systems are remarkably opaque. Vendors keep designs proprietary and offer minimal documentation. Academic researchers have had to reverse engineer their protocols piece by piece to understand how they work [10, 13, 39, 53, 61, 82].

The complexity of analyzing these systems is exacerbated by their inherently conflicting privacy and abuse-prevention goals. On the one hand, privacy of tag owners requires that a tag's Bluetooth advertisements be frequently rotated and *unlinkable*, so that a "super observer" cannot fingerprint the tag and track its owner over time. On the other hand, abuse-prevention requires the opposite: bystander devices must be able to *link* a tag's advertisements in order to detect when a rogue tag is following them. Cryptography helps reconcile these goals, but every vendor solves the problem differently, and the details are rarely public. Academic work has shown attacks against both the privacy [16, 49, 52, 54, 83, 84] and

abuse-prevention [10, 13, 38, 57, 61] implementations. In parallel, researchers have developed formal models to capture the complex security and privacy goals of OF protocols and proposed secure solutions [19, 20, 22, 25, 56].

What has resulted is a fragmented landscape: results are spread across different attack surfaces, adversary models, and mitigations, making it difficult to systematically assess which protections work, which fail, and what design principles should guide future OF systems. Without transparency, it's hard for researchers or policymakers to assess whether the solutions are secure or even functional. This motivates our systematization effort.

Bluetooth Low Energy (BLE). BLE communication occurs via advertisements or full connections. Advertisements are connectionless broadcasts used for discovery and connection initiation. Even after devices have paired/bonded, an initiator can keep advertising without forming a connection, and the peer can reply with lightweight responses to indicate it is in range. When a connection is established, bidirectional data exchange follows the Generic Attribute Profile (GATT) protocol [9].

3 LITERATURE SEARCH METHODOLOGY

To construct our corpus of papers, we used two independent approaches, each conducted by a different author. For the first method, we began with a seed set of 13 highly cited and recent papers that directly addressed the security and privacy of OF systems and examined both backward and forward citations to identify additional relevant work. We recursively did this until reaching saturation. For the second method, we conducted a systematic keyword-based search across major academic venues in security, privacy, and cryptography. The academic venues we searched were USENIX Security, ACM CCS, IEEE S&P, NDSS, PETS, SOUPS, ACM WiSec, ACM WPES, and IACR ePrint from 2014-2025. Our final corpus, available on our repository <https://github.com/cortegap/usenix-26-sok-artifact>, is the union of the two methods, consisting of 40 academic papers and nine industry documents. We defer more details regarding our methodology to Appendix A.

4 PROTOCOL PHASES & IMPLEMENTATIONS

OF systems can be divided into four main sub-protocols: pairing, connected mode, lost mode, and unwanted tracking detection. Here we discuss each of these stages. Along the way we present a canonical construction, BasicOF, that achieves the described functionality and discuss how four major vendors—Apple, Google, Samsung, and Tile—instantiate these phases in their respective implementations.

4.1 Pairing

Pairing is the first step in the life cycle of a tracker tag, where a user pairs a *tracker tag* with one or more *owner devices*. Tracker tags are small, battery-powered BLE devices with limited memory and no internet or GPS capabilities. Owner devices, typically smartphones, share the same account and have internet connectivity.

In this phase, the tag and its owner device (1) authenticate themselves as legitimate participants in the service provider's network and (2) establish a shared key, with some help from the service provider. Intuitively, this can be achieved by executing an Authenticated Key Exchange (AKE) protocol between the tag and the owner

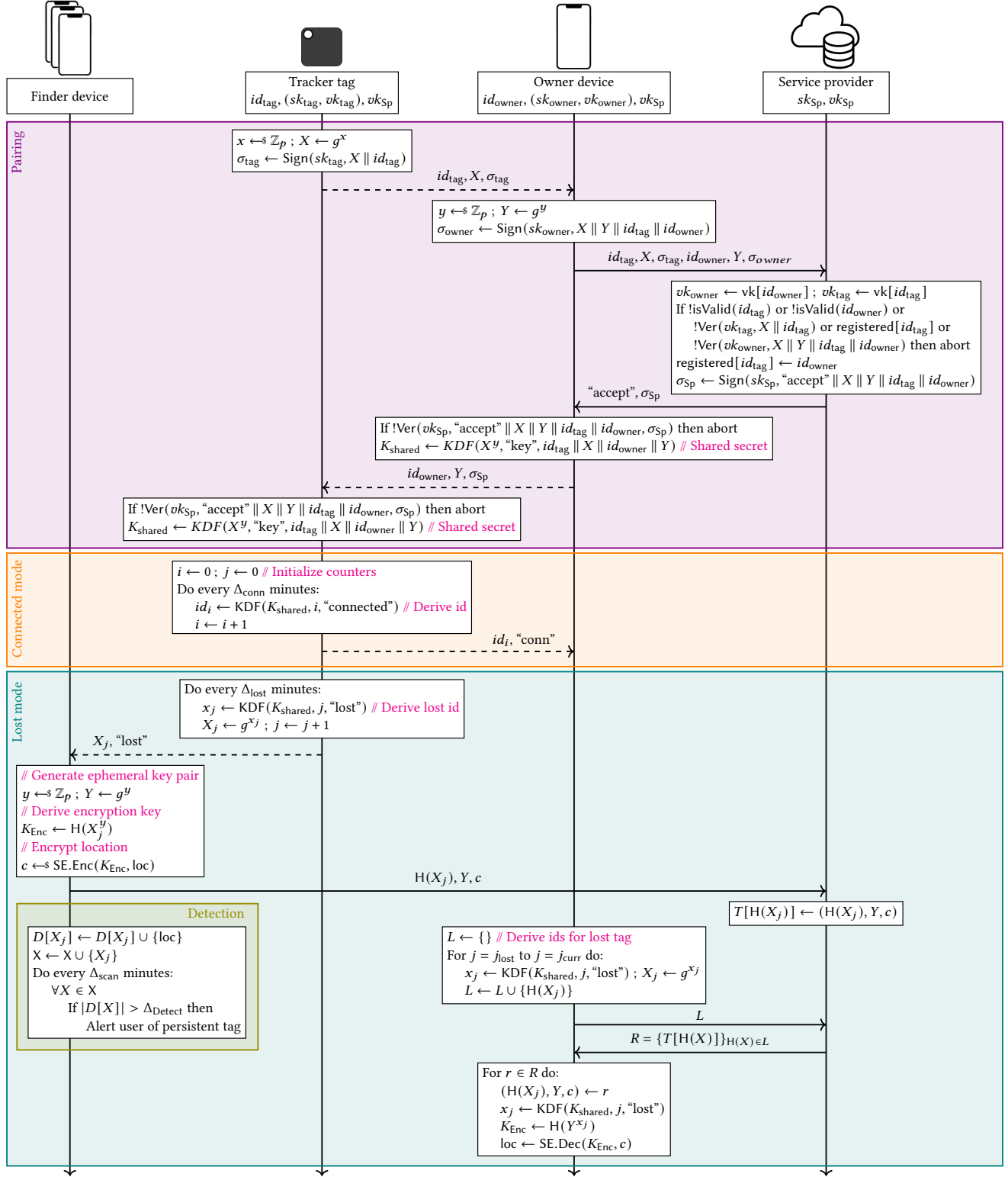


Figure 1: The canonical construction BasicOF for an OF protocol built using an authenticated key exchange protocol AKE, a key derivation function KDF, and a symmetric encryption scheme SE as building blocks. Here, we instantiate AKE using a standard signed Diffie-Hellman key exchange protocol. Dashed arrows denote BLE advertisements; solid arrows denote TLS connections.

	Key Exchange Participants	(Tag) authentication method	Summary of KE
Apple	Tag, owner	Tag authenticates to the server by encrypting its serial number and other secrets provisioned at manufacture time under the server's public encryption key pk_{server} . The server decrypts and checks whether the serial number matches the secrets and ensures that the tag was not previously registered.	The tag and the owner device generate ephemeral key pairs, (x, X) and (y, Y) , and nonces n_{tag} and n_{owner} respectively. The tag and owner then compute the "master" public key $X_0 \leftarrow X \cdot Y$, the connected-mode secret key (SKN_0) and the separated-mode secret key (SKS_0) as $SKN_0 \parallel SKS_0 \leftarrow KDF(X_0, n_{tag} \parallel n_{owner})$. The shared key K_{shared} is (X_0, SKN_0, SKS_0) .
	Tag, owner	Tag only authenticates implicitly to the owner device after pairing by producing a MAC over a nonce n using an intermediate key $K_{account}$ derived during key exchange. The owner device verifies the MAC against a locally generated one to authenticate tag.	The server assigns a key pair $(x_{modelld}, X_{modelld})$ to each tag of the same modelld. The owner samples its ephemeral key pair (y, Y) and computes $K_{DH} \leftarrow H(X_{modelld}^y)$. Then it samples two symmetric keys $K_{account}$ and K_{shared} . It encrypts $K_{account}$ under K_{DH} and $K_{ephemeralld}$ under $K_{account}$, and sends both ciphertexts to the tag. The tag derives K_{DH} and decrypts ciphertexts to get K_{shared} .
Samsung	Tag, server	Tag authenticates to the server by sending a hash of its MAC address. Tag also authenticates to the owner device by producing an encryption of the fixed string "smartthings" under the shared key. The owner decrypts the ciphertext and checks if the message obtained equals "smartthings".	The tag sends $h \leftarrow H_0(macAddress)$ to the owner device which samples a random string $rand$ and sends $(h, rand)$ to the server. The server retrieves the tag's public key X using h , samples its ephemeral key pair (y, Y) , and computes $K_{master} \leftarrow H_1(X^y \parallel rand)$. It sets parameters IV , $poolSize$, and seed and sends K_{master} and $(IV, seed, poolSize, Y)$ to the owner device, which forwards the latter to the tag. The tag computes K_{master} and sets $K_{shared} \leftarrow (K_{master}, IV, seed)$.
	Tag, server	Tag authenticates to the server by providing a unique device identifier and a hash of the model-specific key provisioned at manufacture time. The server verifies the hash and ensures that the tag was not previously registered.	After generating $sresT$, the tag computes the shared key $K_{shared} \leftarrow KDF(K_{interim}, sresT)$. Upon receiving the modelld, the server retrieves the corresponding $K_{interim}$. It derives $sresT$ analogously to the tag and $K_{shared} \leftarrow KDF(K_{interim}, sresT)$. Finally, the server sends K_{shared} to the owner device.
Tile	Tag, server	Tag authenticates to the server by providing a unique device identifier and a hash of the model-specific key provisioned at manufacture time. The server verifies the hash and ensures that the tag was not previously registered.	

Table 1: Comparison of pairing implementations across vendors.

device where the service provider establishes the relevant Public Key Infrastructure (PKI).

Canonical construction. In Figure 1, we present a canonical protocol for the pairing phase of an OF system. We assume that the service provider provisions unique identities, id_{tag} and id_{owner} , and signing key pairs, (sk_{tag}, vk_{tag}) and (sk_{owner}, vk_{owner}) , to the tag and owner device respectively. The service provider has its own signing key pair $(sk_{server}, vk_{server})$, with vk_{server} known to both the tag and the owner. We further assume that, at the start of the pairing protocol, the tag and the owner know each other's identities but not each other's verification keys. This differs from standard PKI settings and could be addressed using provider-issued certificates. For simplicity, our construction omits certificates.

Our protocol involves the execution of a standard signed Diffie-Hellman (DH) AKE protocol to derive a shared secret K_{shared} between the tag and its owner. The tag and the owner each generate ephemeral key pairs (x, X) and (y, Y) respectively. Each party signs their ephemeral public key using their signing key and sends their identity, their ephemeral public key, and the corresponding signature to the server, with the owner device acting as a proxy to the server for the tag. The server verifies the received signatures and ensures that the tag was not previously registered. Upon successful verification, the server maps the tag to the owner as registered pairs in its local database. Finally, the tag and the owner device execute a Diffie-Hellman key exchange to derive K_{shared} using their respective shares X and Y .

Deployed protocols. We now give a high-level overview of the pairing phase implementations of the four major OF vendors we consider. We provide full details for each implementation in Appendix B and a summary in Table 1.

All implementations we study use an AKE protocol to establish a shared key between the tag and the owner device during pairing. In all implementations, the owner device authenticates itself by

requiring the owner to log in to their account with the provider. However, they differ in:

- (1) *Who participates in the KE:* We observe two distinct design choices across implementations. Either the KE is executed directly between the tag and the owner device (Apple and Google), or between the tag and the server (Samsung and Tile). In the latter case, the server additionally sends K_{shared} to the owner device over a TLS connection: all three parties have the secret key.
- (2) *How tags authenticate:* Tags may authenticate themselves to the server via manufacture-provisioned secrets (Apple, Tile, Samsung) or cryptographically to the owner (Google).

These design choices impact both the security and privacy guarantees of the OF system as discussed in Section 6.

4.2 Connected mode

After registration, the tag operates in connected mode when it is within Bluetooth range of its owner device. In this phase, the tag periodically broadcasts BLE advertisements derived from the shared key, which its owner device can recognize and acknowledge.

One may ask: *why would a tag continue advertising after pairing, rather than switching to an encrypted BLE connection?* The answer lies in energy efficiency. Tags are low-cost, coin-cell-powered devices that cannot afford to maintain continuous BLE connections. Instead, they broadcast low-power advertisements and rely on lightweight responses from the owner device to determine proximity. BLE connections between the owner device and tag are only established when performing user actions on the tag, such as Apple's support for instructing a tag to make an audible sound.

BLE advertisements are broadcast unencrypted, making them readable by any attacker with a radio frequency (RF) detector. If a tag uses a static identifier—or a static MAC address—such an attacker can easily fingerprint it and track a user based on the advertisements broadcast by their tag. This is a known issue in Bluetooth privacy and MAC address randomization is a standard

	Connected-mode ID generation	State encoding
Apple	$(X_0, SKN_0, SKS_0) \leftarrow K_{\text{shared}}$ $SKN_i \leftarrow \text{KDF}(SKN_{i-1}, \text{"update"})$ $u_i \parallel v_i \leftarrow \text{KDF}(SKN_i, \text{"diversify"})$ $u_i \leftarrow u_i + 1; v_i \leftarrow v_i + 1$ $id_i \leftarrow X_0^{u_i} \cdot g^{v_i}$	The second bit of the second byte of the payload is set to 1 to indicate connected mode.
Google	$r'_i \leftarrow \text{KDF}(K_{\text{shared}}, 0xFF \parallel \text{rotExp} \parallel i \parallel 0x00 \parallel \text{rotExp} \parallel i)$ $r_i \leftarrow r'_i \bmod n$ $id_i \leftarrow g^{r_i}$	The unwanted tracking byte (UT) encodes a tag's state and is set to 0x40 in connected mode.
Samsung	$(K_{\text{master}}, \text{seed}, IV) \leftarrow K_{\text{shared}}$ $K_{\text{privd}} \leftarrow \text{KDF}_0(K_{\text{master}}, \text{"privacy"})$ $x_i \leftarrow 0 \parallel (i \wedge 255) \parallel \text{seed} \parallel 0 \parallel (i \wedge 255)$ $id_i \leftarrow \text{KDF}_1(K_{\text{privd}}, IV, x_i)[64]$	Tags encode their state in bits 5–7 of the first byte of the advertisement payload. A tag can be in one of six states, based on the number of connected devices or the duration since disconnection from its owner (see Table 12 [82]).
Tile	$K_{\text{seed}} \leftarrow \text{KDF}(K_{\text{shared}}, \text{tile_uuid} \parallel \text{"identity"})$ $id_i \leftarrow \text{KDF}(K_{\text{seed}}, i)[64]$	Tags do not differentiate between modes and hence do not encode state.

Table 2: Comparison of connected-mode implementations across vendors.

mitigation [2, 81]. However, randomizing the MAC address alone is insufficient if the payload contains a static identifier. So, tags periodically rotate their identifiers, such that they are unlinkable.

Canonical construction. We present a canonical protocol for connected-mode interactions in Figure 1. Our construction derives connected-mode identifiers id_i by applying a key-derivation function KDF to the shared key K_{shared} and an epoch counter i . The counter is incremented every Δ_{conn} minutes; this is a configurable parameter of the scheme. Connected-mode advertisements also encode their mode—as indicated by the string “conn” in the advertisement—signaling to nearby finder devices that the tag is in connected mode and that its location should not be reported.

Deployed protocols. We detail the connected-mode identifier-generation methods for the four vendors in Table 2. All implementations set some epoch duration Δ_{conn} and derive their connected-mode advertisements by applying a KDF to (a function of) K_{shared} . They differ in exactly what function of K_{shared} is used to derive the identifiers id_i and how the state of the tag is encoded in these advertisements. Though Apple and Google’s connected-mode identifiers are public keys, they are never used for encryption. Apple and Google reserve a single bit in advertisements to indicate the tag’s mode. Samsung tags can advertise one of six different operating states, based on how many devices the tag is connected to and how long it has been disconnected (see Table 12 of [82]). Tile does not distinguish between connected and lost modes.

4.3 Lost mode

Once a tag has been outside Bluetooth range of its owner device for a threshold duration, it transitions to lost mode. In this state, the tag periodically broadcasts BLE advertisements intended for discovery by nearby finder devices—bystander devices in the service provider’s network—that have both GPS and internet capabilities and continuously scan for such advertisements. Upon receiving a lost-mode advertisement, a finder extracts a special lost-mode identifier therein, computes its current GPS location, and sends the

identifier and its current location to the service provider. To locate the lost tag, the owner can subsequently query the provider with its tag’s identifiers and retrieves associated reports. We summarize these interactions in Figure 2.

Canonical protocol. We present a canonical protocol for lost-mode interactions in Figure 1. It uses an epoch duration Δ_{lost} , which determines how frequently a tag rotates its lost-mode identifier. As we discuss in Section 4.4, Δ_{conn} and Δ_{lost} are distinct to accommodate the differing privacy goals of the two modes. A lost tag derives ephemeral secret key x_j by applying a key-derivation function KDF to the shared key K_{shared} and the epoch counter j , and it then generates the corresponding public key as $X_j \leftarrow g^{x_j}$. Additionally, lost-mode advertisements encode this state, as indicated by the string “lost” in the advertisement. While the derivation is structurally similar to connected mode, it is context-separated.

Upon receiving a lost-mode advertisement, a finder sees the tag’s ephemeral public key X_j , generates its own ephemeral key pair (y, Y) , obtains its current GPS location loc , and constructs a location report $c \leftarrow \text{Enc}(K, \text{loc})$ where $K \leftarrow H(X_j^y)$. It then sends $H(X_j)$, Y , and c to the service provider.

To locate their lost tag, the owner device determines a range $[j_{\text{lost}}, j_{\text{curr}}]$ of epoch counters covering when the tag was lost, computes X_j for $j \in [j_{\text{lost}}, j_{\text{curr}}]$ using K_{shared} , and queries the service provider for associated reports by sending each $H(X_j)$. The provider responds with any matching reports, which the owner decrypts using x_j to recover the tag’s location loc .

Deployed protocols. Apple’s derivation of lost-mode identifiers is similar to the connected-mode derivation given in Table 2, except it uses a separated-mode key SKS_0 instead of SKN_0 . Moreover, it unsets the second bit of the second byte of the payload. Google and Samsung use the same identifier generation as in the connected mode, but set the UT byte to 0x41 (Google) and the 3-bit operational state to 010 or 011 depending on how long the tag was disconnected from its owner (Samsung). Apple, Samsung, and Google all set Δ_{lost} to 24 hours in the lost mode. As mentioned in Section 4.2, Tile does not differentiate between lost and connected modes.

Apple and Google require finders to end-to-end encrypt location information for lost tags by default. In these systems the lost-mode identifiers are public encryption keys. Finders encrypt their location under the advertised public key, and submit the encryption and the hash of the public key to the service provider. The owner decrypts reports by deriving the appropriate decryption key from K_{shared} .

Samsung offers end-to-end encryption as an opt-in feature. The owner sets a 6-digit PIN, generates encryption keys (pk_e, sk_e) , encrypts sk_e under the hash of the PIN and an IV , producing ciphertext c , and sends (pk_e, c, IV) to the server. Tags with E2EE enabled encode that fact in a designated field of the advertisement. Finders request pk_e from the server using the tag’s identifier, encrypt the location under pk_e , and send it with the identifier to the server. The owner decrypts the reports using sk_e .

Tile does not end-to-end encrypt location reports. Finder devices send id_j and the location in plaintext (over TLS) to the service.

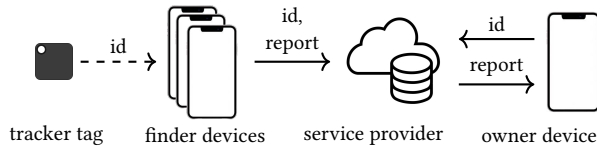


Figure 2: A summary of the lost mode of an OF protocol. Dashed arrows denote BLE advertisements; solid arrows denote TLS connections.

4.4 Unwanted tracking detection

The phases of an OF protocol we have described so far assume that users of the protocol are benign. However, tracker tags have aided several criminal activities including theft, stalking, assault, and even murder [8, 15, 17, 28, 55, 71, 72]. In this setting, a malicious user surreptitiously places their own tag on their target’s person or belongings and uses the network to monitor the target’s movements. To combat tracker-aided stalking, manufacturers have deployed unwanted tracking detection or “anti-stalking” mechanisms that attempt to alert a user if an unknown tag has been following them.

Intuitively, this requires a finder device to link advertisements broadcast by a tag over time. However, this goal directly conflicts with the unlinkability property we discussed in Section 4.2. A common approach to address this tension is to introduce distinct epoch durations, Δ_{conn} and Δ_{lost} , for the connected and lost modes respectively, such that $\Delta_{\text{conn}} < \Delta_{\text{lost}}$. This means that tags rotate their identifiers more frequently in connected mode than in lost mode. As a result, in lost mode, a finder device can more easily link identifiers across locations to detect tracking, while in connected mode the faster rotation protects the owner’s privacy against RF adversaries. In effect, implementations trade off some privacy against RF adversaries in the lost mode to allow unwanted tracking detection.

Canonical protocol. We present a canonical protocol for unwanted tracking detection in Figure 1. Recall that in this construction, finder devices run periodic Bluetooth scans for lost-mode advertisements. In addition to reporting the location of a lost tag, finder devices run a detection algorithm. This algorithm maintains a database of observed lost-mode identifiers and the locations where they were recorded. If the same identifier is recorded at more than a threshold number of distinct locations, the finder device’s user is alerted of potential unwanted tracking.

Deployed protocols. All major OF vendors provide some form of unwanted tracking detection. Apple provides *Item Safety Alerts*, Google provides *Unknown Tracker Alerts*, Samsung offers *Unknown Tag Detection*, and Tile provides its *Scan and Secure* feature. At their core, Apple, Google, and Samsung follow the same basic approach as in Fig. 1 with Δ_{lost} set to 24 hours, running detection scans periodically in the background at the OS level. However, individual implementations may enforce additional heuristics like time of day, battery level, and connection to cellular/WiFi network to optimize their algorithms. These heuristics are opaque, owing to the proprietary nature of the systems, and have not been reverse-engineered in academic work, so we omit them from our description.

Tile’s approach differs fundamentally in that it requires the user to manually initiate a Bluetooth scan. The scan requires the finder

device to be in motion for about 10 minutes, during which six consecutive Bluetooth scans are run. The device records all observed Tile advertisements and forwards them to the server, which then returns identifying information about the detected tags.

5 OF SECURITY AND PRIVACY

In this section, we provide a synthesis of the security and privacy goals discussed in the literature. This synthesis also helps us highlight goals that are missing and that we believe future work should address. We will group these into two sets of goals, what we refer to as baseline and additional. In Section 6, we use these properties as a framework for evaluating the security of deployed OF implementations as well as academic proposals. First, though, we detail the different adversaries and their capabilities that our security definitions address.

Adversaries and their capabilities. As we will see, a wide variety of security properties have or can be targeted by OF systems. Underlying them are a smaller number of threat models related to adversarial capabilities, which we distill down into the following:

- *RF adversary (active/passive):* An attacker equipped with a radio frequency (RF) detector who can passively intercept Bluetooth transmissions within range or actively inject arbitrary Bluetooth messages.
- *Service provider:* Since OF systems rely on centralized infrastructure, we consider the service provider itself as a potential adversary capable of passively observing or actively manipulating protocol interactions.
- *Malicious tag:* An attacker who fabricates counterfeit tracker tags that behave similarly to provider-approved tags. These tags exploit the OF network for tracking while evading detection mechanisms intended to prevent misuse such as stalking.
- *Malicious owner:* An attacker who owns a legitimate tag but deviates from the protocol and exploits the OF network to learn more information than it is privy to.
- *Temporary physical access:* An attacker who gains temporary physical access to a tag and can extract the tag’s secrets.

Note that we have omitted standard network adversaries (e.g., on-path compromised routers) because deployed protocols use TLS for point-to-point communications. We allow for collusion between adversaries. We define the threat model for the security goals defined in the following section based on these adversary definitions.

The complete set of goals appears in Table 3, where for each goal, we list the corresponding adversary, and give a security definition. We begin by discussing each definition in the baseline set.

Tag-owner binding ensures that a tag is paired with exactly one owner account throughout its life cycle. Without this property, a malicious owner could re-register an already registered tag to themselves, allowing them in the worst case, to track the original owner without their knowledge.

Unlinkability ensures that a passive/active RF adversary or the service provider cannot fingerprint a tag by correlating its broadcasts over time. In its absence, an attacker could track an owner based on the advertisements broadcast by its tag. For example, an

	Security Goal	Adversary	Definition
Baseline	Tag-owner binding [25, 53, 56]	Malicious owner	It is infeasible for an attacker that controls multiple owner accounts to register the same tag simultaneously under more than one account.
	Unlinkability [19, 20, 22, 52, 53, 56]	Passive RF, active RF, and service provider	Any two BLE advertisements broadcast by the same tag that are at least Δ time apart are indistinguishable from advertisements broadcast by two different tags.
	Location confidentiality [19, 22, 53, 56]	Service provider	Any two location reports r_0 and r_1 generated by a finder device for distinct locations (ℓ_0, t) and (ℓ_1, t) are indistinguishable.
	Location report integrity [19]	Service provider	It is infeasible for an attacker to modify honestly generated location reports without being detected.
	Detectability [22]	Malicious owners	A finder device can efficiently determine and alert the user of suspected stalking if a sequence of lost-mode advertisements $\{\text{adv}_0, \text{adv}_1, \dots, \text{adv}_{\Delta_{\text{detect}}}\}$ recorded at distinct locations $(\ell_0, t_0), (\ell_1, t_1), \dots, (\ell_{\text{detect}}, t_{\text{detect}})$ originated from the same tag.
Additional	Finder privacy [25]	Service provider, malicious owner	Any two location reports submitted by the same finder device are indistinguishable from reports submitted by two distinct finder devices.
	Forward secrecy [19]	Temporary physical access	It is infeasible for an attacker to compromise the location confidentiality of reports submitted before tag compromise.
	Post-compromise security [19]	Temporary physical access	It is infeasible for an attacker to compromise the location confidentiality of reports submitted after tag compromise.
	Finder proximity [25]	Malicious finders	It is infeasible for an attacker to submit any report for a tag at location (ℓ, t) and have it be accepted by the corresponding owner without actually having been in the tag's proximity at time t .
	Stalker identity integrity	Malicious owners and malicious tags	A finder device can efficiently retrieve the identity of the owner of a tag that was detected for potential stalking.
	Advertisement unforgeability [19, 56]	Malicious tags, malicious owners	It is infeasible for an attacker to forge advertisements that are indistinguishable from ones produced by legitimate tags and retrieve location reports for them.

Table 3: Security goals for Offline Finding protocols and the corresponding threat model.

attacker could place an RF receiver near the target's home and monitor recorded Bluetooth advertisements to learn whether the target is at home. In the worst case, the attacker could set up a network of RF detectors and surveil the movements of large populations.

Location confidentiality ensures the service provider does not learn the locations of the finder or the reported tag. In its absence, the service provider effectively runs a mass-surveillance network.

Location report integrity prevents the service provider from modifying location reports submitted by finders.

Detectability provides potential stalking victims a reliable way to link advertisements from a tag over time to recognize when an unknown tag is following them. Without detectability, a malicious owner could place a tag on their target's person or belongings and use the OF network to monitor their movements. We note here that we define detectability against malicious owners only, not malicious tags i.e. our definition does not capture the failure to detect malicious tags. To capture security against malicious tags, we introduce the advertisement unforgeability property under additional goals.

Additional properties. We now discuss additional properties, that are not explicit goals for deployed implementations and have not received much academic attention, but ones we believe are important to improve the privacy of and reduce the misuse of OF networks. These additional properties enhance three high-level goals: (1) protecting the location privacy of finders (finder privacy), (2) protecting location data even under compromise of tag secrets (forward secrecy and post-compromise security), and (3) holding abusers of the

network accountable (finder proximity, stalker identity integrity, and advertisement unforgeability).

Finder privacy prevents malicious owners and the service provider from correlating different location reports submitted by the same finder. Without it, the malicious owner or the service provider could deanonymize finders and infer their movements based on the reports they submit. For instance, an attacker could place several tags along a particular geographic area. By correlating the location reports submitted for different tags by the same finder, the attacker could reconstruct the finder's trajectory. Although vendors often claim that finders in their network remain anonymous, these claims have not been rigorously analyzed or formalized.

Forward secrecy ensures that even if an adversary compromises a tag's secrets, it cannot retroactively learn the tag's past location from reports submitted before tag compromise. **Post-compromise security** similarly ensures that an adversary cannot learn tag locations from reports submitted after tag compromise. Together, forward secrecy and post-compromise security prevent long-term privacy loss from a single compromise: the attacker learns at most a small window of locations, not the tag's full history.

Finder proximity ensures that only finders physically near a lost tag can submit any reports for it. Without it, malicious finders could remotely inject false reports for tags of their choice and poison the network. Such an adversary could, for example, generate "evidence" that an honest user's tag was seen at a particular location, thereby framing them for being complicit in stalking.

Stalker identity integrity enables a user that has been a target of unwanted tracking to—in collaboration with a judge or the service provider—retrieve the identity of the owner of the tag. Then, the service provider and law enforcement can take appropriate action against the perpetrator. Without this property, victims might detect they are being tracked but have no way to identify or prosecute the perpetrator.

Advertisement unforgeability ensures that only tags approved by the provider can generate valid advertisements that owners can later use to retrieve their tag's location. It particular, it prevents attackers from deploying counterfeit tags that remain compatible with the OF protocol but deliberately deviate from it in order to evade unwanted tracking detection. Mayberry et al. [56] formalized this notion as beacon unforgeability and proposed a construction based on partially blind signatures to achieve it.

5.1 Unlinkability v/s anti-abuse

Unlinkability and the anti-abuse properties (detectability and stalker identity integrity) are inherently in tension. The former requires that broadcasts hide identifying information that the latter requires to be revealed in some circumstances. As discussed in Section 4.4, some deployed protocols balance this tension by using distinct modes of operation—connected and lost—for the tag. In the connected mode, the tag changes its identifier every $\Delta_{\text{conn}} = 15$ minutes and in the lost mode every $\Delta_{\text{lost}} = 24$ hours. So in the lost mode, such protocols only achieve unlinkability for Δ_{lost} which is strictly weaker than unlinkability achieved in the connected mode. This deliberate weakening of achieved unlinkability allows users to detect stalking.

6 SECURITY AND PRIVACY ANALYSIS

In this section, we use the definitions from Table 3 to evaluate the security of four major deployed protocols and five constructions proposed in academic papers. We begin with the BasicOF protocol described in Fig. 1 and provide intuition for why it satisfies all baseline security goals. Our intention is to demonstrate that a canonical protocol built from standard cryptographic building blocks achieves all the baseline security goals. Then we evaluate deployed protocols. For each property a protocol fails to achieve, we detail the relevant attacks or design choices, citing prior work where applicable. Then we present the academic constructions, describing the set of goals they aim to achieve and providing intuition for how they achieve them. Table 4 provides a summary.

In Table 4, we evaluate unlinkability guarantees in both connected and lost modes by separately analyzing unlinkability for $\Delta = 15$ minutes and $\Delta = 24$ hours. Protocols that rely on distinct modes of operation to balance the privacy–detectability tension should therefore always achieve unlinkability for $\Delta = 24$ hours, and achieve unlinkability for $\Delta = 15$ minutes in connected mode.

The BasicOF construction. Recall that BasicOF is constructed by canonically composing an authenticated key exchange protocol (AKE), a key derivation function (KDF), and a symmetric encryption scheme (SE). Tag-owner binding follows from the (authentication) security of AKE. Unlinkability follows from the key indistinguishability security of AKE and the PRF security of KDF. Location confidentiality and integrity follow from the IND-CPA and IND-CTXT

security of SE. Detectability follows from slower identifier rotation—trading off unlinkability—in the lost mode, allowing finder devices to check if the same lost-mode identifier (i.e., the same X_j) has been recorded at multiple locations over time.

We note that BasicOF makes blackbox use of KDF and SE but instantiates AKE with a variant of the signed Diffie-Hellman protocol. This reflects the fact that the pairing phase in OF cannot be captured using standard AKE syntax. While the latter is executed between two parties who mutually authenticate each other and derive a shared secret, the former involves three parties: the tag, the owner device, and the server, where the tag and owner authenticate to the server while establishing a shared key with each other.

Deployed protocols often implement this phase in ways that introduce vulnerabilities. The concrete instantiation of AKE in BasicOF serves to show that, in principle, the required security guarantees can still be achieved using known techniques.

6.1 Deployed protocols

Tag-owner binding. Apple and Tile achieve this by cryptographically authenticating the tag and the owner device during pairing, then storing a map between the owner account and the tag in their databases (see Section 4.1). AirTags can only be paired to one account; resetting the AirTag does not allow its registration with a new owner. Tile allows for device re-registration after reset. However, resetting a tag removes the associated owner-tag mapping.

Google fails to provide tag-owner binding because the server does not authenticate tags at registration; tags are registered in Find Hub against a set of precomputed advertisements. An attacker can therefore record another user's tag advertisements and re-register them as a "fake" tag (location report denial-of-service attack described in Section 6.5 of [13]).

Samsung also fails to achieve tag-owner binding. Samsung attempts authenticating tags during registration, but their implementation is vulnerable to the following attack. An attacker can reset an already registered tag (say, tag_1), forcing it into the pairing phase. Because Samsung's key exchange occurs between the server and the tag, and a tag's long-term secret key is derived deterministically from its MAC address, a malicious owner device can mount a man-in-the-middle attack (attack V5 of [53]), impersonating a new tag (tag_2) to the server. The server records the attacker as the owner of tag_2 , but the attacker actually pairs with tag_1 , which was already registered to another user, breaking tag-owner binding.

Unlinkability ($\Delta = 15\text{min}$). Apple achieves 15-minute unlinkability from the server but not from RF adversaries and only in connected mode. Recall that during pairing, the tag transmits (x, n_{tag}) , where x is the tag's ephemeral secret and n_{tag} a random nonce; the phone responds with (Y, n_{owner}) . Rotating keys are derived from $pk_0 = Y \cdot g^x$, n_{tag} , and n_{owner} . An RF observer of the pairing can derive pk_0 , and thus track advertisements. This attack (V7 in [53]) can be mounted passively by forwarding messages between the tag and the owner.

Google achieves unlinkability from RF adversaries but not from the server. Owners must upload precomputed advertisements so that the server can match location reports to owners; hence the server can link advertisement values to the owner.

Security Goal		Deployed protocol						Academic proposal					
		BasicOF	Apple	Google	Samsung (default)	Samsung (E2EE)	Tile	PrivateFind [80]	SECROW [25]	Eddystone-FID [20]	BlindMy [56]	EBGHJ24 [22]	BenNtf [19]
Baseline	Tag-owner binding	●	●	○	○	○	●	△	●	○	●	○	△
	Unlinkability [$\Delta = 15$ minutes]	●	●	●	○	○	○	●	○	●	●	●	●
	Unlinkability [$\Delta = 24$ hours]	●	●	●	○	○	○	●	○	●	●	●	●
	Location confidentiality	●	●	●	○	●	○	●	●	○	●	●	●
	Location report integrity	●	●	●	○	●	○	●	●	○	●	●	●
	Detectability	●	●	○	●	●	●	○	○	○	●	●	○
Additional	Finder Privacy	○	○	△	△	△	△	●	●	○	○	○	△
	Forward secrecy	○	●	○	○	○	○	○	○	○	○	○	●
	Post-compromise security	○	●	○	○	○	○	○	○	○	○	○	●
	Finder Proximity	○	○	○	○	○	○	●	●	○	○	○	○
	Stalker identity integrity	○	●	○	○	○	●	○	○	○	○	○	○
	Advertisement unforgeability	○	○	○	○	○	○	○	△	○	●	○	○

Table 4: Summary of protocols and their achieved security properties. We use ● to denote that a protocol achieves a property, ● to denote that a property is partially achieved (i.e., only against certain adversaries or in specific modes of operation), and ○ to indicate that a property is not achieved. We use △ to indicate that it is unclear whether a protocol achieves a given property either because it needs more analysis of the protocol or because the protocol is underspecified.

Samsung and Tile do not achieve unlinkability against either adversary. In Samsung, the server stores the shared master key, and advertisements recycle through 1000 values, allowing an RF adversary to passively capture and link advertisements. In E2EE mode, finders who query the server for a tag’s long-term public key can fingerprint the tag. Meanwhile, Tile uses static MAC addresses.

Unlinkability ($\Delta = 24$ h). Apple achieves 24-hour unlinkability from the server, but not from RF adversaries due to the RF attack described above. Google provides 24-hour unlinkability from RF adversaries, but not from the server for reasons listed above. Samsung and Tile fail for the same reasons as in the 15-minute case.

Location confidentiality and integrity. Finders in Apple, Google, and Samsung (E2EE mode) encrypt location reports end-to-end using an authenticated encryption scheme. Tile and Samsung (default mode) do not encrypt location data.

Detectability. Recall that the detectability notion only considers legitimate tags. Apple and Samsung achieve it through automatic background scanning, and Tile only partially satisfies it since scans must be triggered manually.

Google fails to achieve detectability. Google’s Find Hub relies on the server to control tags’ transition to the lost mode: a tag only enters lost mode if the server has not received recent reports from its owner. An attacker can evade detectability by repeatedly submitting fake location reports for a tag, thereby preventing its transition to lost mode and hence its detection.

Takeaway: None of the deployed protocols achieve all baseline goals due to: (i) intentional design decisions (e.g., Samsung and Tile do not encrypt location reports), (ii) tensions between security properties (e.g., Apple and Google trade off unlinkability for detectability), (iii) vulnerable cryptographic implementation (e.g., Apple’s and Samsung’ key exchange protocols), or (iv) threats outside of their models (e.g., Google does not aim for unlinkability against the server).

Finder privacy. None of the deployed protocols provide finder privacy against the service provider, since finders must authenticate to the provider before submitting reports. Apple and Google explicitly claim finder privacy against malicious owners [34, 42, 43]. Additionally, Apple fails to provide finder privacy against tag owners through an attack in which a malicious owner can recover sub-trajectories of reporting devices by placing AirTags at intersections of an urban grid [73]. The attack exploits a server-side side channel which the owner can then use to correlate finder reports.

By contrast, the extent of finder privacy in Google, Samsung, and Tile systems remains largely unexplored. In principle, providers could include metadata in finder reports that allow owners to link multiple reports to the same finder. Because we were unable to independently verify or rule out such practices, we conservatively mark the corresponding entries in our evaluation as inconclusive.

Forward secrecy and post-compromise security. Apple and Samsung achieve forward secrecy and post-compromise security. In Apple’s protocol, a tag learns only the master public key during pairing, not the corresponding secret key. As a result, if an adversary compromises a tag, they can reconstruct all past and

future public keys used to encrypt location reports, but not the decryption keys. In the Samsung protocol's E2EE mode, the key used for encrypting location reports is generated and stored by the owner, so tag compromise does not break location confidentiality of past or future reports. By contrast, Google's protocol establishes a shared symmetric key between the tag and its owner, from which both tag identifiers, encryption and decryption keys are derived. Compromising a tag's secrets would allow the adversary to decrypt all past and future location reports. Tile trivially fails to provide either property—it fails to provide location confidentiality at all.

Finder proximity. No deployed protocol achieves finder proximity. If an attacker knows a tag's identifier (by observing it or via an unlinkability attack), they can submit arbitrarily many location reports for the identifier without its presence.

Stalker identity integrity. Apple, Google, and Tile each provide mechanisms for a stalking victim to recover partial identity of a detected tag's owner (e.g. first four digits of phone number or characters of an email address). Apple's approach is non-cryptographic: once the user locates the detected tag, they extract the tag's unique serial number and query Apple's server, which returns partial account information for the owner. Because the server maintains a binding between serial numbers and owners, the integrity of the returned identity is preserved. Google implements a similar process, allowing the victim to retrieve identifying information and query the server for the corresponding owner's identity. Tile also claims to support stalker identification, but unlike Apple and Google, users cannot retrieve the information themselves—victims are required to involve law enforcement, after which Tile claims to provide the owner's identity. To the best of our knowledge, Samsung does not support any such mechanism.

Advertisement unforgeability. None of the deployed protocols achieve this property, since they all use standard advertisement formats and pseudorandom identifiers. An attacker can thus forge advertisements that are indistinguishable from genuine tag broadcasts. Prior work has shown that such attacks enable unauthorized use of the OF network and evasion of unwanted-tracking detection [7, 10, 12, 13, 38, 57].

Takeaway: For most of the additional security goals, deployed protocols either do not achieve them or it is unclear whether they do. The former is because these goals fall outside the protocol's stated threat model and the latter because the available documentation is underspecified.

6.2 Academic proposals

Here, we describe and evaluate six academic proposals using our framework. Each proposal addresses only a subset of the goals defined in Table 3. Accordingly, we focus on the specific goals each proposal aims to achieve. We also discuss efficiency constraints that hinder practical deployment.

PrivateFind [80]. PrivateFind achieves unlinkability, location confidentiality and integrity, finder privacy, and finder proximity. The pairing phase of PrivateFind establishes K_{shared} between the tag and the owner. After pairing, the tag and owner derive ephemeral

identifiers as $id_{i+1} \leftarrow \text{KDF}(K_{\text{shared}}, id_i)$ every 15 minutes, achieving 15-minute unlinkability. Tags become discoverable only when out of Bluetooth range of the owner. A finder detecting a lost tag sends its location to the tag, which encrypts it under K_{shared} and returns it to the finder, who forwards the ciphertext to the server. As K_{shared} is only known to the owner and tag, location confidentiality and integrity are ensured.

PrivateFind achieves finder privacy by design: finders are not required to register accounts with the provider and the server omits any metadata in its reports. Thus, neither a malicious owner nor the service provider can identify which finder submitted a given report. Finally, as a finder must interact with the tag over BLE to report a location and replay attacks are prevented using counters in the AEAD scheme, PrivateFind also achieves finder proximity. Deployment challenge. Requiring tags to be interactive in lost mode significantly increases their computation and communication overhead, thereby reducing battery life.

SECROW [25]. SECROW achieves tag-owner binding, location confidentiality and integrity, finder proximity, and finder privacy. Each tag and owner device is provisioned a long-term signing key pair and a unique identifier; owner and finder devices additionally have a Trusted Execution Environment (TEE) that can generate attested ephemeral signing keys. During pairing, both the tag and the owner device authenticate themselves to the service provider using their long term private keys and establish K_{shared} . This together with the fact that the server stores a tags-to-owners mapping guarantees tag-owner binding.

The tag's static identifier is accessible to finders. During the location-reporting phase, a finder authenticates to the service provider using one of its ephemeral signing keys generated by the TEE, hence achieving finder privacy. The server returns a nonce, which the finder forwards to the tag along with its own location. The tag encrypts the location under K_{shared} using an authenticated encryption scheme, signs the server's nonce, and returns the ciphertext, signature, its identifier to the server via the finder. The server verifies the signature and stores the report. As locations are encrypted with K_{shared} , SECROW achieves location confidentiality and integrity. Moreover, since location reporting requires active interactions with the tag over BLE, it achieves finder proximity.

Deployment challenge. The security of PrivateFind requires a TEE on all owner and finder devices, limiting large scale deployment.

Eddystone-EID [20]. Eddystone-EID is designed for a setting that is slightly different from that of OF systems: in addition to an identifier, tags broadcast some telemetry data (e.g., battery status, temperature). The protocol guarantees unlinkability of identifiers and confidentiality of the tag's telemetry data.

During pairing, the tag and the owner establish two symmetric keys: one for identifier generation and another for end-to-end encryption of telemetry data. The owner device shares the identifier generation key with the server. The server uses this to create an identifier-to-owner mapping by generating all identifiers for a tag. After pairing, the tag broadcasts ephemeral identifiers generated by applying a KDF to the identifier-generation key and the current epoch along with the AEAD encryption of its telemetry data under the end-to-end encryption key. Finders append their location to

the tag's broadcast and forward them to the server, which in turn forwards it to the intended owner. Since the location data is not encrypted, it does not guarantee location confidentiality or integrity. While Eddystone-EID does not introduce deployment challenges, we note that it does not achieve all baseline security goals.

BlindMy [56]. Apple's unwanted-tracking alerts were shown to be easily bypassed by programming a malicious tag to emulate multiple AirTags, producing lost messages that nearby finders would accept and report [57]. An attacker could then query these reports to track a victim. BlindMy cryptographically prevents such exploits by achieving advertisement unforgeability, while also providing unlinkability, location confidentiality and integrity, and detectability—properties otherwise achieved in the same way as Apple's protocol.

BlindMy uses partially blind signatures to achieve advertisement unforgeability. It modifies Apple's pairing and location retrieval phase. During pairing, in addition to establishing K_{shared} as in Apple's protocol, the owner precomputes all identifiers that the tag will broadcast over its lifetime. Then, the owner device sends the hash of these identifiers ("blinded" so the server does not learn them) along with their corresponding epochs to the server. The server verifies that the epochs are valid and produces signatures over each (blinded identifier hash, epoch) pair and returns these signatures to the owner device. The owner device unblinds the signatures and stores all (identifier, unblinded signature) pairs, and shares all the identifiers with the tag.

When the owner requests reports, it sends the signed identifier hashes and epochs to the server. The server verifies these before returning matching reports, ignoring any invalid requests. The owner then decrypts the reports analogously to Apple's protocol. Deployment challenge. The pairing and location-report-download phases are significantly different from those of existing protocols, making their integration non-trivial.

EBGHJ24 [22]. As pointed out in Section 4.4, unlinkability and detectability are inherently conflicting goals and typically, deployments trade off unlinkability in the lost mode for detectability. The OF protocol proposed in [22], that we refer to as EBGHJ24, improves this trade-off; it simultaneously guarantees strong 15-minute unlinkability and detectability. It also achieves location confidentiality and integrity by following the same steps as Apple.

To achieve strong unlinkability and privacy, [22] introduce a new cryptographic primitive called multi-dealer secret sharing (MDSS). Analogous to traditional secret sharing [67]—where a dealer splits a secret into shares such that a threshold number of them can recover the initial secret—MDSS sets a threshold for reconstruction. MDSS additionally guarantees that even when multiple dealers share different secrets, a receiver can recover these secrets from a set of mixed shares. Using MDSS as a building block, [22] propose an OF construction that achieves both unlinkability and detectability. Deployment challenge. While [22] implemented their construction on hardware to demonstrate feasibility, the advertisement size in their construction requires the tag to broadcast twice as many times as in Apple's protocol, reducing its battery life by half.

BcnNtf [19]. BcnNtf achieves unlinkability against RF adversaries (but not the server), location confidentiality and integrity, and both forward and post-compromise security. The protocol also defines

an integrity notion which is similar to advertisement unforgeability in Table 3 but does not consider security against malicious owners.

BcnNtf operates in the same setting as Eddystone-EID, but instead of separately broadcasting its identifier and encrypted telemetry, BcnNtf encapsulates telemetry within the identifier itself. The identifier also serves as a public key, which finders use to encrypt location data, thereby ensuring confidentiality and integrity.

To achieve forward and post-compromise security, [19] introduces XDHIES, an extension of the DHIES [1] scheme that provides inherent forward and backward security. Crucially, BcnNtf separates keys for identifier generation from those for decrypting location reports, and tags only learn the former. Then, tag compromise reveals public keys but not decryption keys, preserving confidentiality and integrity of reports, as in Apple's protocol.

Deployment challenge BcnNtf deviates significantly from existing deployments to achieve advanced properties, making it hard to achieve backwards compatible integration.

Takeaway: Academic proposals individually achieve many of the additional goals that deployed protocols fail to achieve. In principle, combining these techniques could yield a construction that achieves the full set of desired properties. However, the proposals either do not meet the battery and bandwidth constraints of tags deployed in practice or introduce compatibility issues with current implementations, rendering deployment infeasible.

7 CHALLENGES BEYOND THE PROTOCOL LEVEL

In this section we describe works highlighting some additional risks and challenges beyond the protocol layer.

BLE security analyses. Even if a protocol achieves unlinkability at the cryptographic or protocol level, the physical layer often leaks side channels that allow efficient and reliable fingerprinting of devices. Characteristics such as static identifiers in advertisements, radio signal patterns, timing behavior, and hardware design choices can all be exploited to link broadcasts or infer device identities. A growing body of work shows that these physical-layer attacks are not only practical but also scalable, undermining unlinkability guarantees by revealing long-term identifiers or persistent correlations across broadcasts. We summarize these works here.

BLE advertisements include a static UUID that enables finders to filter devices for OF purposes, but also exposes a fingerprinting vector. Zuo et al. [84] demonstrated large-scale device fingerprinting using wide-area scanning, identifying Tile among the top 10 device types. Moreover, attackers can exploit signal strength of scanned advertisements to further distinguish devices of the same type.

Ludant et al. [54] showed that current Bluetooth chip design enables linking BLE advertisements to Bluetooth Classic (BTC) frames through their shared clock, revealing the device's global identifier (BDADDR). The attack required BLE and BTC to be simultaneously used, which is quite common (e.g. a user using an AirTag as well as their AirPods). Tested against Apple's Find My, this attack linked ephemeral tag identifiers to user identities. More generally, this physical-layer vulnerability lets attackers correlate tag broadcasts even when protocols aim to provide unlinkability.

Zhang and Lin [83] presented another BLE fingerprinting attack that is effective despite randomized MACs and payloads. The attack exploits communication patterns between a paired peripheral and central when allow-lists are used for automatic reconnection. Because the central and peripheral change MAC addresses asynchronously, an attacker can link these patterns to continuously track the peripheral over long periods.

Leu et al. [49] demonstrated a distance-reduction attack against the Ultra-Wideband (UWB) ranging feature in Apple's AirTags, allowing an adversary to make a phone believe the tag is nearby when it is not. While the implications for the security notions we consider are not immediately clear, this result highlights how the threat landscape may expand as UWB becomes more integrated into OF systems.

David et al. [52] showed yet another failure of unlinkability in practice through a novel physical-layer vulnerability they call the Battery Insertion Attack. In this attack, an adversary can restart a beacon at a chosen time and leverage the resulting timing patterns between identifier rotations to link broadcasts, even when periodic rotation is used. The key insight is that attackers observe not only the identifiers themselves but also the precise moments at which they change. Since existing unlinkability definitions do not model timing information, the authors propose a stronger notion—timed-sequence indistinguishability—which accounts for both the content and timing of broadcasts.

Chen et al. [16] demonstrate a large-scale privacy attack that exploits a BLE vulnerability in Apple's Find My network. The vulnerability arises because, although the Find My protocol specifies that tags in lost mode must broadcast using static random addresses, this requirement is difficult to enforce in practice. As a result, the network also accepts broadcasts using other address types, namely non-resolvable and resolvable private addresses. The attack allows adversaries to remotely transform ordinary devices—including desktops, laptops, smartphones, and IoT devices running Linux, Windows, or Android—into trackable “tags”, making it possible to uniquely locate them over the long term.

Unwanted tracking detection in practice. While we discussed earlier detectability definitions, here we focus on prior work that reversed engineered currently deployed anti-stalking mechanisms, proposed new ones, and strived to understand the stalking risks of tracking devices through user and measurement studies.

Prior work investigated the broader IoT ecosystem risks and analyzed the ways how these technologies can be abused. Stephenson et al. [69] performed a measurement study on websites about how abusers use IoT devices, including Bluetooth trackers, for interpersonal abuse. Meanwhile, Wei et al. [79] did a measurement study on TikTok videos that give anti-privacy and anti-security advice on using technology for surveillance and control in interpersonal relationships. Stephenson et al. [68] conduct a series of semi-structured interviews to people with first or second-hand experience with IoT abuse to understand how survivors identify abuse, strategies to mitigate it, and challenges in doing so. Finally, Ceccio et al. [14] performed a survey of spy devices and detection tools available, showing how spy devices are easy to acquire, but hard to detect with existing mechanisms.

Some work looked into understanding how anti-stalking tools available specifically for tracking devices work. Heinrich et al. [37] reverse-engineered Apple's anti-stalking protection mechanism. They point out how Apple's protocol at the time was not sufficient to protect the users from being stalked, since notifications were delivered too late, when user's private locations, like their home, has most likely already been revealed to the abuser. Turk et al. [76] describe the mechanisms available to detect AirTags, SmartTags, Tile, and Chipolo trackers. The point out weaknesses in such mechanisms, ranging from how hard to use, to the challenge in distinguishing between honest and malicious usage of these trackers. Shafquat et al. [66] investigate AirTags anti-stalking mechanism. The work considers how different factors, such as device model, local time, etc. play a role in the time it takes to trigger safety alert.

Additionally, some previous work showed attacks against current anti-stalking protocols. Shafquat et al. [66] show an attack where tampering with some bytes in AirTags BLE advertisements lead to trackers not triggering safety alert notifications. Mayberry et al. [57] show how to create tracking tags that participate in the Find My network, but that do not trigger safety alerts, based on attacks that involve bit flipping and rotating keys faster. These attacks, combined with the ability of creating fake or cloned tags, as shown in [12, 13, 38, 61], display great deficiencies in the anti-stalking tools available to users.

Given the weaknesses of manufacturer-produced anti-stalking tools, previous work also builds new applications and techniques to detect malicious trackers. AirGuard [37] is an Android app for detecting tags following the Find My protocol. This included detection for fake tags using the OpenHaystack framework [38]. It was proposed due to a gap between the ability of Apple and Android devices in detecting AirTags. Meanwhile, BLE-Doubt [11] is an Android app to detect trackers from multiple vendors. HomeScout [58] extends AirGuard to detect Tile and SmartTags too and BL(u)E CRAB [18] include an application to improve trackers detection by relying on more factors beyond time and distance. Finally, Despres et al. [21] propose an algorithm to detect trackers even when these rotate MAC addresses.

At last, several works performed user studies to analyze the usability of the anti-stalking features. Heinrich et al. [40] analyzed data from AirGuard users and conducted an online survey among such users about the misuse of Bluetooth trackers and protections against it. Turk and Hutchings [75] conducted a user experiment based on the Assassins game to study how easy it is to use trackers and how usable anti-stalking features are. Gerhardt et al. [26, 27] ran a user study looking into when participants receive an anti-stalking notification, to evaluate the reliability of unwanted trackers, and how users react to receiving these notifications.

8 DISCUSSION

In this section, we reflect on the lessons learned from our analysis. We highlight recurring challenges, tensions between privacy and abuse-prevention, and gaps in current designs that leave room for both abuse and unintended privacy harms. We identify five main themes that we believe should guide future work on OF systems.

Privacy versus abuse prevention. Unlinkability has emerged as one of the most difficult properties to achieve in practice. Even

though most protocols explicitly aim to achieve unlinkability, both protocol-level and physical-layer attacks repeatedly break this property. Unlinkability often complicates stalker detection, sometimes creating more opportunities for abuse than it prevents.

Achieving unlinkability and detectability simultaneously at the protocol level is challenging, with many designs sacrificing one to preserve the other. Among the systems we evaluated, only the construction of [22] achieves both properties, but at the cost of higher communication overhead on the tag, making it impractical for real-world deployment. Unlinkability is further undermined by physical-layer vulnerabilities in BLE and Bluetooth that allow efficient tag fingerprinting. These attacks are difficult to mitigate due to specification complexity, wide deployment, and legacy constraints.

Future work suggestion: We suggest that detectability is a critical and pressing property to target for improvements given the ongoing, widespread use of tags for stalking. Ideally, the community would work to develop global standards for detectability of any location-enabled device that could be used for surreptitious tracking, building off and improving nascent efforts [23, 48]. Towards informing such standards, future work could focus on new design points that target better detectability while preserving, or even modestly relaxing, current unlinkability mitigations.

Accountability. Detectability without accountability is insufficient. Despite their concerning use for harm, most OF protocols lack accountability features that would enable reporting or banning misbehaving users.

Achieving accountability requires a reporting mechanism that lets a victim notify the platform or law enforcement about a detected tag. The platform/law enforcement can then work with the victim to identify the tag's owner. This is technically challenging due to conflicts with other goals. For example, a mechanism for retrieving a stalker's identity could also potentially allow the service provider or law enforcement to deanonymize owners of arbitrary tags, compromising privacy.

Future work suggestions: We see potential opportunity in exploring cryptographic accountability mechanisms. Prior work on message franking—a privacy-preserving accountability mechanism for end-to-end encrypted messengers—demonstrates that privacy-accountability tensions can be balanced using cryptographic tools [36, 47, 77], and cryptographic proof-of-stalking extensions for OF protocols seems a plausible future direction. In addition, future work should conduct further research on non-cryptographic accountability mechanisms: reporting that uses static hardware-defined identifiers, digital stalking reports that cannot be forged if the tag's software is intact, and supporting procedures and policies.

Interoperability. A recurring challenge across all protocols we studied is the lack of interoperability by design. Before Apple and Google drafted the DULT [48] specification, each vendor's ecosystem was siloed: a user could only detect unwanted trackers if they belong to the same vendor network. While DULT has improved cross-vendor detection between Android and Apple, vendors like Tile are still confined to only being able to detect their own tags.

Future work suggestions: For user safety, we argue that interoperability must be a design priority. This means that we need internet standards, akin to TLS, that should be adopted by all potentially privacy-damaging devices—not just tracker tags, but also other

wireless beacons and emerging IoT hardware. Building towards this vision would require generic constructions, and cooperation across vendors, regulators, and standards bodies. Ideally this would end up win-win: in addition to privacy and safety improvements, interoperable finder networks would improve functionality.

Formal analyses. Several cryptographic analyses of OF protocols exist, but most focus on a subset of properties in isolation. This is understandable given the complexity of these systems, which pursue multiple, often orthogonal, goals. Isolating individual properties allows for fine-grained analysis, but leaves open whether guarantees hold when protocols are modeled and analyzed comprehensively.

In addition, some properties such as tag-owner binding do not map neatly onto existing cryptographic primitives. Classical abstractions from key-exchange security are insufficient, since OF introduces a unique paradigm that involves not only the tag and owner but also the server, which authenticates entities.

Future work suggestions: We call for the development of new cryptographic models and abstractions tailored to OF protocols, as well as analyses that explicitly address how these properties interact and compose in practice.

OF extensions for data back-hauling. A small but growing body of work explores how OF networks can be repurposed for opportunistic data backhaul. Prior works demonstrate that Apple's Find My can support arbitrary data muling [7, 65] as well as distributed sensing via crafted BLE advertisements [35]. A more recent academic proposal, Nebula [78], explicitly addresses the privacy risks of such piggybacking, introducing a decentralized architecture that prevents the service provider from tracing relay devices while enabling payments, compensation, and spam protections.

Future work suggestions: As the scope of data piggybacked onto OF-like networks expands, so too do the associated privacy threats. Future work should carefully analyze how these systems compose and how the privacy risks inherent in OF networks may propagate to higher-level protocols and applications.

9 LIMITATIONS

Our analysis is based on the academic literature and vendor specifications/statements. Literature predating 2021 does not capture subsequently deployed protocols. Our discussion of the four major protocols reflects their current state: (i) academic analysis of these protocols is recent (Apple:2021-2025, Samsung:2024-2025, Google:2025, Tile:2026) and (ii) most attacks identified result from intentional design choices or hardware vulnerabilities, rather than implementation mistakes. In doing our analysis we searched for statements made by vendors about changes to their respective protocols. There is no public evidence that vendors have changed their protocol designs substantively, and we performed some experimentation with Google and Tile to validate our understanding of the protocols. However, we did not experimentally reconfirm details for all deployed protocols.

Relatedly, our assertions that a protocol achieves a given property are supported by prior analyses. Academic proposals include security proofs, which we used to guide Table 4. For deployed protocols, prior work provides either provable security analysis (e.g., [56] proves tag-owner binding, location confidentiality and integrity,

and unlinkability of Apple's protocol, [20] proves location confidentiality and integrity of Google's protocol) or formal verification (e.g., [53] proved the location confidentiality and integrity of Samsung's protocol when E2EE is enabled). We did not perform independent provable security or formal analysis to reconfirm results.

ACKNOWLEDGEMENT

Generative AI-based tools were used to improve flow and correct typos, grammatical errors, and awkward phrasing.

REFERENCES

- [1] Michel Abdalla, Mihir Bellare, and Phillip Rogaway. DHAES: An encryption scheme based on the diffie-hellman problem. *Cryptology ePrint Archive*, Paper 1999/007, 1999.
- [2] Mohammad Afaneh. Bluetooth addresses & privacy in bluetooth low energy, 2020.
- [3] Apple. One app to find it all. <https://www.apple.com/icloud/find-my/>.
- [4] Apple. Find my network accessory specification. https://www.frandroid.com/wp-content/uploads/2020/06/Find_My_network_accessory_protocol_specification.pdf, 2020. Accessed: 2025-04-13.
- [5] Apple. Legal process guidelines. <https://www.apple.com/legal/privacy/law-enforcement-guidelines-us.pdf>, 2025. Accessed: 2025-04-13.
- [6] Apple. What to do if you get an alert that an airtag, set of airpods, find my network accessory, or compatible bluetooth location-tracking device is with you. <https://support.apple.com/en-us/119874>, 2025. Accessed: 2025-04-13.
- [7] Alex Bellon, Alex Yen, and Pat Pannuto. Tagalong: A free, wide-area data-muling service built on the airtag protocol. In *Proceedings of the 20th ACM Conference on Embedded Networked Sensor Systems*, SenSys '22, page 782–783, New York, NY, USA, 2023. Association for Computing Machinery.
- [8] Lindsey Bever. She tracked her boyfriend using an airtag — then killed him, police say, Jun 2022.
- [9] Bluetooth Special Interest Group (SIG). Generic attribute profile (gatt). Bluetooth Core Specification v5.4, Part F, 2022.
- [10] Fabian Bräunlein. Send my: Arbitrary data transmission via apple's find my network.
- [11] Jimmy Briggs and Christine Geeng. Ble-doubt: Smartphone-based detection of malicious bluetooth trackers. In *2022 IEEE Security and Privacy Workshops (SPW)*, pages 208–214, 2022.
- [12] Lukas Burg, Max Granzow, Alexander Heinrich, and Matthias Hollick. Openhaystack mobile - tracking custom find my accessories on smartphones. In *Proceedings of the 15th ACM Conference on Security and Privacy in Wireless and Mobile Networks*, WiSec '22, page 277–279, New York, NY, USA, 2022. Association for Computing Machinery.
- [13] Leon Böttger, Alexander Matern, Dennis Arndt, and Matthias Hollick. Okay google, where's my tracker? security, privacy, and performance evaluation of google's find my device network. *Proceedings on Privacy Enhancing Technologies*, 2025.
- [14] Rose Ceccio, Sophie Stephenson, Varun Chadha, Danny Yuxing Huang, and Rahul Chatterjee. Sneaky spy devices and defective detectors: The ecosystem of intimate partner surveillance with covert devices. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 123–140, Anaheim, CA, August 2023. USENIX Association.
- [15] Hartley Charlton. Apple's airtag item trackers increasingly linked to criminal activity, Dec 2021.
- [16] Junming Chen, Xiaoyue Ma, Lannan Luo, and Qiang Zeng. Tracking you from a thousand miles away! turning a bluetooth device into an apple airtag without root privileges. In *Proceedings of the 34th USENIX Security Symposium*, 2025. Short Presentation, Distinguished Artifact Award Winner.
- [17] Samantha Cole. Police records show women are being stalked with apple airtags across the country, Apr 2022.
- [18] Dylan Conklin, Primal Pappachan, and Roberto Yus. Bl(u)e crab: A user-centric framework for identifying suspicious bluetooth trackers. In *2025 IEEE International Conference on Pervasive Computing and Communications Workshops and other Affiliated Events (PerCom Workshops)*, pages 570–572, 2025.
- [19] Liron David, Omer Berkman, Avinatan Hassidim, David Lazarov, Yossi Matias, and Moti Yung. Extended Diffie-Hellman Encryption for Secure and Efficient Real-Time Beacon Notifications. In *2025 IEEE Symposium on Security and Privacy (SP)*, pages 4406–4418, Los Alamitos, CA, USA, May 2025. IEEE Computer Society.
- [20] Liron David, Avinatan Hassidim, Yossi Matias, Moti Yung, and Alon Ziv. Eddystone-eid: Secure and private infrastructure protocol for ble beacons. *IEEE Transactions on Information Forensics and Security*, 17:3877–3889, 2022.
- [21] Tess Despres, Noelle Davis, Prabal Dutta, and David Wagner. Detagrive: Linking macs to protect against malicious ble trackers. In *Proceedings of the Second Workshop on Situating Network Infrastructure with People, Practices, and Beyond*, 2023.
- [22] Harry Eldridge, Gabrielle Beck, Matthew Green, Nadia Heninger, and Abhishek Jain. Abuse-resistant location tracking: Balancing privacy and safety in the offline finding ecosystem. In *USENIX Security Symposium*, 2024.
- [23] C. Fossaceca and E. Rescorla. Finding tracking tags. IETF working group, 2024.
- [24] Christian Frank, Philipp Bolliger, Christof Roduner, and Wolfgang Kellerer. Objects calling home: locating objects using mobile phones. In *Proceedings of the 5th International Conference on Pervasive Computing*, Pervasive '07, page 351–368, Berlin, Heidelberg, 2007. Springer-Verlag.
- [25] Chinmay Garg, Aravind Machiry, Andrea Continella, Christopher Kruegel, and Giovanni Vigna. Toward a secure crowdsourced location tracking system. In *Proceedings of the 14th ACM Conference on Security and Privacy in Wireless and Mobile Networks*, WiSec '21, page 311–322, New York, NY, USA, 2021. Association for Computing Machinery.
- [26] Daniel Gerhardt, Matthias Fassl, Carolyn Guthoff, Adrian Dabrowski, and Katharina Krombholz. Airtag-facilitated stalking protection: Evaluating unwanted tracking notifications and tracker locating features. In *34th USENIX Security Symposium (USENIX Security 25)*, Seattle, WA, August 2025. USENIX Association.
- [27] Daniel Gerhardt, Matthias Fassl, and Katharina Krombholz. Poster: Will our colleagues detect the airtag? let's check (consensually). 2024.
- [28] Thomas Germain. Alleged stalking victims accuse tile of advertising its devices as women trackers, Aug 2024. Accessed: 2024-10-21.
- [29] Mark Gibbs. Phone halo's tracker devices make it harder to lose your stuff. *Network World*, November 2013.
- [30] Google. Find hub. <https://www.google.com/android/find/>.
- [31] Google. 5 ways to use android's new find my device, 2024.
- [32] Google. Find hub network accessory specification (fmdn). <https://developers.google.com/nearby/fast-pair/specifications/extensions/fmdn>, 2025. Google Fast Pair documentation.
- [33] Google. Google fast pair service: Introduction. <https://developers.google.com/nearby/fast-pair/specifications/introduction>, 2025. Google Fast Pair documentation.
- [34] Google. How find hub protects your data. <https://support.google.com/android/answer/14796936?hl=en-en>, 2025.
- [35] Max Granzow, Alexander Heinrich, Matthias Hollick, and Marco Zimmerling. Poster: Leveraging apple's find my network for large-scale distributed sensing. In *Proceedings of the 22nd Annual International Conference on Mobile Systems, Applications and Services*, MOBISYS '24, page 666–667, New York, NY, USA, 2024. Association for Computing Machinery.
- [36] Paul Grubbs, Jiahui Lu, and Thomas Ristenpart. Message franking via committing authenticated encryption. In *Advances in Cryptology – CRYPTO*, 2017.
- [37] Alexander Heinrich, Niklas Bittner, and Matthias Hollick. Airguard - protecting android users from stalking attacks by apple find my devices. In *Proceedings of the 15th ACM Conference on Security and Privacy in Wireless and Mobile Networks*, 2022.
- [38] Alexander Heinrich, Milan Stute, and Matthias Hollick. Openhaystack: a framework for tracking personal bluetooth devices via apple's massive find my network. In *Proceedings of the 14th ACM Conference on Security and Privacy in Wireless and Mobile Networks*, 2021.
- [39] Alexander Heinrich, Milan Stute, Tim Kornhuber, and Matthias Hollick. Who can find my devices? security and privacy of apple's crowd-sourced bluetooth location tracking system. *Proceedings on Privacy Enhancing Technologies*, 2021.
- [40] Alexander Heinrich, Leon Würsching, and Matthias Hollick. Please unstalk me: Understanding stalking with bluetooth trackers and democratizing anti-stalking protection. *Proceedings on Privacy Enhancing Technologies*, 2024.
- [41] Apple Inc. Apple introduces airtag, 2021.
- [42] Apple Inc. Find my security. <https://support.apple.com/en-in/guide/security/sec6cbc80fd0/1/web/1>, 2021.
- [43] Apple Inc. Apple platform security guide. https://help.apple.com/pdf/security/en_US/apple-platform-security-guide.pdf, 2024.
- [44] Jon Keegan and Alfred Ng. The popular family safety app life360 is selling precise location data on its tens of millions of users, Dec 2021.
- [45] Jon Keegan and Alfred Ng. There's a multibillion-dollar market for your phone's location data, Sep 2021.
- [46] Akshaya Kumar, Anna Raymaker, and Michael Specter. Security and privacy analysis of tile's location tracking protocol. <https://arxiv.org/abs/2510.00350>, 2025.
- [47] Junzuo Lai, Gongxian Zeng, Zhengang Huang, Siu Ming Yiu, Xin Mu, and Jian Weng. Asymmetric group message franking: Definitions and constructions. In Carmi Hazay and Martijn Stam, editors, *Advances in Cryptology – EUROCRYPT 2023*, pages 67–97, Cham, 2023. Springer Nature Switzerland.
- [48] Brent Ledvina, David Lazarov, Ben Detwiler, and Siddika Parlak Polatkan. Detecting unwanted location trackers accessory protocol. IETF working group, 2024.
- [49] Patrick Leu, Giovanni Camurati, Alexander Heinrich, Marc Roeschlin, Claudio Anliker, Matthias Hollick, Srdjan Capkun, and Jiska Classen. Ghost peak: Practical distance reduction attacks against HRP UWB ranging. In *31st USENIX Security Symposium*, 2022.

- Symposium (USENIX Security 22)*, pages 1343–1359, Boston, MA, August 2022. USENIX Association.
- [50] life360. Tile tracker. <https://www.life360.com/tile-trackers>.
 - [51] Life360. How to handle unwanted tracking. <https://support.life360.com/hc/en-us/articles/30582987898135-How-to-Handle-Unwanted-Tracking>, 2022.
 - [52] David Liron, Avinatan Hassidim, Yossi Matias, and Moti Yung. The battery insertion attack: Is periodic pseudo-randomization sufficient for beacon privacy? *Proceedings on Privacy Enhancing Technologies*, 2025.
 - [53] Xiaofeng Liu, Chaoshun Zuo, Qinsheng Hou, Pengcheng Ren, Jianliang Wu, Qingchuan Zhao, and Shanjing Guo. A thorough security analysis of ble proximity tracking protocols. In *34th USENIX Security Symposium (USENIX Security 25)*. USENIX Association, August 2025.
 - [54] Norbert Ludant, Tien D. Vo-Huu, Sashank Narain, and Guevara Noubir. Linking bluetooth le & classic and implications for privacy-preserving bluetooth-based protocols. In *2021 IEEE Symposium on Security and Privacy (SP)*, pages 1318–1331, 2021.
 - [55] Ryan Mac and Kashmir Hill. Are apple airtags being used to track people and steal cars?, Dec 2021.
 - [56] Travis Mayberry, Erik-Oliver Blass, and Ellis Fenske. Blind my—an improved cryptographic protocol to prevent stalking in apple’s find my network. *Proceedings on Privacy Enhancing Technologies*, 2023.
 - [57] Travis Mayberry, Ellis Fenske, Dane Brown, Jeremy Martin, Christine Fossaceca, Erik C Rye, Sam Teplov, and Lucas Foppe. Who tracks the trackers? circumventing apple’s anti-tracking alerts in the find my network. In *Proceedings of the 20th Workshop on Workshop on Privacy in the Electronic Society*, 2021.
 - [58] Katharina O. E. Müller, Louis Bienz, Bruno Rodrigues, Chao Feng, and Burkhard Stiller. Homescout: Anti-stalking mobile app for bluetooth low energy devices. In *2023 IEEE 48th Conference on Local Computer Networks (LCN)*, pages 1–9, 2023.
 - [59] Inc. Nut Technology. Nutfind: Official online store for nut brand smart tracker series, 2025.
 - [60] Sarah Perez. Tile, the lost-item tracker with millions in crowdfunding, was worth the wait, June 2014.
 - [61] Thomas Roth, Fabian Freyer, Matthias Hollick, and Jiska Classen. Airtag of the clones: Shenanigans with liberated item finders. In *2022 IEEE Security and Privacy Workshops (SPW)*. IEEE, 2022.
 - [62] Samsung. Smartthings find. <https://www.samsung.com/levant/apps/smartthings-find/>.
 - [63] Samsung. How to disable smarttag 2 location tracking? <https://www.samsung.com/ae/support/mobile-devices/how-to-disable-smarttag-2-location-tracking/>, 2025. Accessed: 2025-08-19.
 - [64] Ltd. Samsung Electronics Co. Introducing the new galaxy smarttag+: The smart way to find lost items, 2021.
 - [65] Spandhana Sara, Moteen Shah, Dhairya Shah, Rajashekar Reddy Chinthalapani, and Ambuj Varshney. Gatehaul: A gateway architecture using backhauling networks to address the connectivity challenges of embedded systems. In *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking, ACM MobiCom '24*, page 1811–1813, New York, NY, USA, 2024. Association for Computing Machinery.
 - [66] Narmeen Shafqat, Nicole Gerzon, Maggie Van Nortwick, Victor Sun, Alan Mislove, and Aanjan Ranganathan. Track you: A deep dive into safety alerts for apple airtags. *Proceedings on Privacy Enhancing Technologies*, 2023.
 - [67] Adi Shamir. How to share a secret. *Commun. ACM*, 22(11):612–613, November 1979.
 - [68] Sophie Stephenson, Majed Almansoori, Pardis Emami-Naeini, and Rahul Chatterjee. "it's the equivalent of feeling like you're in Jail": Lessons from firsthand and secondhand accounts of IoT-Enabled intimate partner abuse. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 105–122, Anaheim, CA, August 2023. USENIX Association.
 - [69] Sophie Stephenson, Majed Almansoori, Pardis Emami-Naeini, Danny Yuxing Huang, and Rahul Chatterjee. Abuse vectors: A framework for conceptualizing IoT-Enabled interpersonal abuse. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 69–86, Anaheim, CA, August 2023. USENIX Association.
 - [70] Google Support. Find unknown trackers. <https://support.google.com/android/answer/13658562?hl=en>.
 - [71] CBS Chicago Team. Man killed girlfriend after she removed airtag he'd secretly placed in her car, prosecutors say, Jul 2023.
 - [72] FOX 7 Austin Digital Team. Texas man uses apple airtag to track down stolen truck, then shoots, kills suspect: police, Apr 2023.
 - [73] Leonardo Tonetto, Andrea Carrara, Aaron Yi Ding, and Jörg Ott. Where is my tag? unveiling alternative uses of the apple findmy service. In *2022 IEEE 23rd International Symposium on a World of Wireless, Mobile and Multimedia Networks (WoWMoM)*, pages 396–405, 2022.
 - [74] Cube Tracker. Cube tracker: Real-time gps tracking solutions, 2025.
 - [75] Kieron Ivy Turk and Alice Hutchings. Stop following me! evaluating the malicious uses of personal item tracking devices and their anti-stalking features. In *Proceedings of the 2024 European Symposium on Usable Security, EuroUSEC '24*, page 277–289, New York, NY, USA, 2024. Association for Computing Machinery.
 - [76] Kieron Ivy Turk, Alice Hutchings, and Alastair R Beresford. Can't keep them away: The failures of anti-stalking protocols in personal item tracking devices. In *Cambridge International Workshop on Security Protocols*, pages 78–88. Springer, 2023.
 - [77] Nirvan Tyagi, Paul Grubbs, Julia Len, Ian Miers, and Thomas Ristenpart. Asymmetric message franking: Content moderation for metadata-private end-to-end encryption. In Alexandra Boldyreva and Daniele Micciancio, editors, *Advances in Cryptology – CRYPTO 2019*, pages 222–250, Cham, 2019. Springer International Publishing.
 - [78] Jean-Luc Watson, Tess Despres, Alvin Tan, Shishir G. Patil, Prabal Dutta, and Raluca Ada Popa. Nebula: A privacy-first platform for data backhaul. In *2024 IEEE Symposium on Security and Privacy (SP)*, pages 3184–3202, 2024.
 - [79] Miranda Wei, Eric Zeng, Tadayoshi Kohno, and Franziska Roesner. Anti-Privacy and Anti-Security advice on TikTok: Case studies of Technology-Enabled surveillance and control in intimate partner and Parent-Child relationships. In *Eighteenth Symposium on Usable Privacy and Security (SOUPS 2022)*, pages 447–462, Boston, MA, August 2022. USENIX Association.
 - [80] Mira Weller, Jiska Classen, Fabian Ulrich, Denis Waßmann, and Erik Tews. Lost and found: stopping bluetooth finders from leaking private information. In *Proceedings of the 13th ACM Conference on Security and Privacy in Wireless and Mobile Networks, WiSec '20*. ACM, July 2020.
 - [81] Martin Woolley. Bluetooth technology protecting your privacy, 2015.
 - [82] Tingfeng Yu, James Henderson, Alwen Tiu, and Thomas Haines. Security and privacy analysis of samsung’s Crowd-Sourced bluetooth location tracking system. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 5449–5466, Philadelphia, PA, August 2024. USENIX Association.
 - [83] Yue Zhang and Zhiqiang Lin. When good becomes evil: Tracking bluetooth low energy devices via allowlist-based side channel and its countermeasure. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, CCS '22*, page 3181–3194, New York, NY, USA, 2022. Association for Computing Machinery.
 - [84] Chaoshun Zuo, Haohuang Wen, Zhiqiang Lin, and Yinqian Zhang. Automatic fingerprinting of vulnerable ble iot devices with static uuids from mobile apps. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, CCS '19*, page 1469–1483, New York, NY, USA, 2019. Association for Computing Machinery.

A METHODOLOGY (EXTENDED)

We now describe the methodology we used to construct our corpus of prior work on offline finding systems and our inclusion criteria. The corpus is included in our artifact, available at <https://anonymous.4open.science/r/usenix-26-sok-artifact-09FF>.

Inclusion criteria. To be included in our study, a paper must have satisfied at least one of the following: (1) directly analyzed the security, privacy, or safety of OF systems or vendor-specific devices (e.g., AirTags, Tiles, SmartTags), (2) contributed broadly to the understanding of adversarial models, security definitions, or abusability concerns in OF systems, (3) proposed secure constructions for or based on bluetooth-based proximity tracking protocols.

Search methodology. To increase coverage, we used two independent approaches, each conducted by a different author.

Method 1: Snowball sampling from seed papers. We began with a seed set of 13 highly cited and recent papers that directly addressed the security and privacy of OF systems. From each paper, we examined both backward and forward citations to identify additional relevant work. A cited paper was added to the corpus if it satisfied our inclusion criteria. We repeated this process iteratively until reaching saturation—that is, no new papers meeting the criteria were discovered in subsequent citation rounds. Our citation-based snowballing approach provided breadth of venue coverage. The corpus resulting of this method is available in our artifact, in `method1-snowball.csv`.

Method 2: Targeted keyword search across venues. To complement the snowballing approach, we conducted a systematic keyword-based search in July 2025 across major academic venues in security, privacy, and cryptography. We searched for papers using the following keywords: *Find My*, *Airtag*, *Tile tracker*, *Find Hub*, *Chipolo*, *Pebblebee*, *SmartTag*, *offline finding*, *safety alert*, *Bluetooth tracking*, *stalkerware*, *crowdsourced network*, *proximity finding*, *finder device*, *location tracking*, *anti-stalking*, *tracking tag*, *BLE advertisements*, and variants thereof. The full list of keywords is available in our artifact, in `keywords.csv`.

The academic venues we searched were

Security/Privacy Conferences: USENIX Security, ACM CCS, IEEE S&P, NDSS, PETS, SOUPS.

Workshops and Specialized Venues: ACM WiSec, ACM WPES

Preprint Archives: IACR ePrint.

We used venue-specific databases for our search: Google Scholar (USENIX, NDSS, PETS, SOUPS), the ACM Digital Library (CCS, WiSec, WPES), and IEEE Xplore (IEEE S&P). This search surfaced 231 papers, which we manually reviewed and filtered down based on our inclusion criteria. The list of 231 papers is available in our artifact, in `method2-keywords.csv`.

Constructing the final corpus. After having an initial list of papers, the authors responsible for each method categorized the papers in their respective corpus. We explain the categorization below. This categorization helped further reducing both corpuses. Additionally, unpublished papers were discarded. For all papers that were found by both approaches and was included by one and excluded by the other (5 papers), the first and second authors met and discussed until reaching consensus. Additionally, all papers

included though one of the methods and not found by the other were reviewed by the author responsible for the latter.

We additionally included (1) the Detecting Unwanted Location Tracking IETF draft(s), (2) official specifications and privacy policies for Apple’s Find My (deprecated at time of writing) and Google’s Find Hub protocols, and (3) support guides explaining what to do in case of unwanted tracking by Apple, Google, Samsung, and Tile. Our final corpus is composed of 49 documents and is included, along with the categorization, in our artifact, in `final-corpus.csv`.

Categorization. Selecting the categories was an iterative process, that included several discussions between the authors responsible for methods 1 and 2, and multiple rounds of re-categorization of papers. Here we describe the final categories. We sorted the papers in the final corpus based on topics and type. For topics papers were sorted among one or more of the following categories: physical layer, OF core protocol, anti-stalking, OF piggybacking, OF-inspired crowd-sourced systems, and privacy policy. For types, the categories were: protocol proposal, attack, formalization, measurement study, user study, reverse engineering, and legal document.

B PAIRING PROTOCOL IMPLEMENTATIONS

B.1 Pairing for Apple’s FindMy

We summarize the pairing protocol for Apple’s Find My network in Figure 3 and describe it below. For brevity, we group certain steps together. Our description is based on the reverse engineering efforts of Heinrich et al. [39] and Liu et al. [53], and Apple’s now depreciated Find My specification [4]. The owner device authenticates itself to the service provider by requiring the user to log in to their iCloud account.

Manufacture-time setup. At manufacture time, each AirTag is provisioned a unique 16-byte serial number, a 1024-byte software token, and a 16-byte software UUID. Additionally, the tag is also given the server’s public encryption key pk_{server}^{Enc} and the server’s public verification key pk_{server}^{Sign} . The Apple server maintains a database that maps each tag’s serial number to its corresponding software token and software UUID.

Tag authentication. As the first step of the pairing process, the owner device samples a uniformly random 32-byte nonce n and generates an 89-byte EncryptedBlob1 and sends both to the tag.¹ The tag generates its ephemeral keys (sk_{tag}^e, pk_{tag}^e) , its own 32-byte nonce n_{tag} , and a 32-byte random seed $seedT$. Then it generates a commitment $C1$ by hashing sk_{tag}^e and n_{tag} . Subsequently, the tag creates a message M as `deviceInfo || EncryptedBlob1 || authToken` where `deviceInfo` contains its serial number, software token, and software UUID. The tag then encrypts M under the server’s public key pk_{server}^{Enc} to produce EncryptedBlob2. Finally, the tag sends EncryptedBlob2 to the owner device.

The owner device now samples its ephemeral key pair $(sk_{owner}^e, pk_{owner}^e)$, and produces commitment $H1$ by hashing pk_{owner}^e and n_{owner} . The owner device forwards EncryptedBlob2 and $H1$ to the server. The server decrypts EncryptedBlob2 to get M and checks that the serial

¹Neither Apple’s specification [4] nor [53] specify what is encrypted within EncryptedBlob1 and we were not able to independently verify what it encrypts so we omit the corresponding details here.

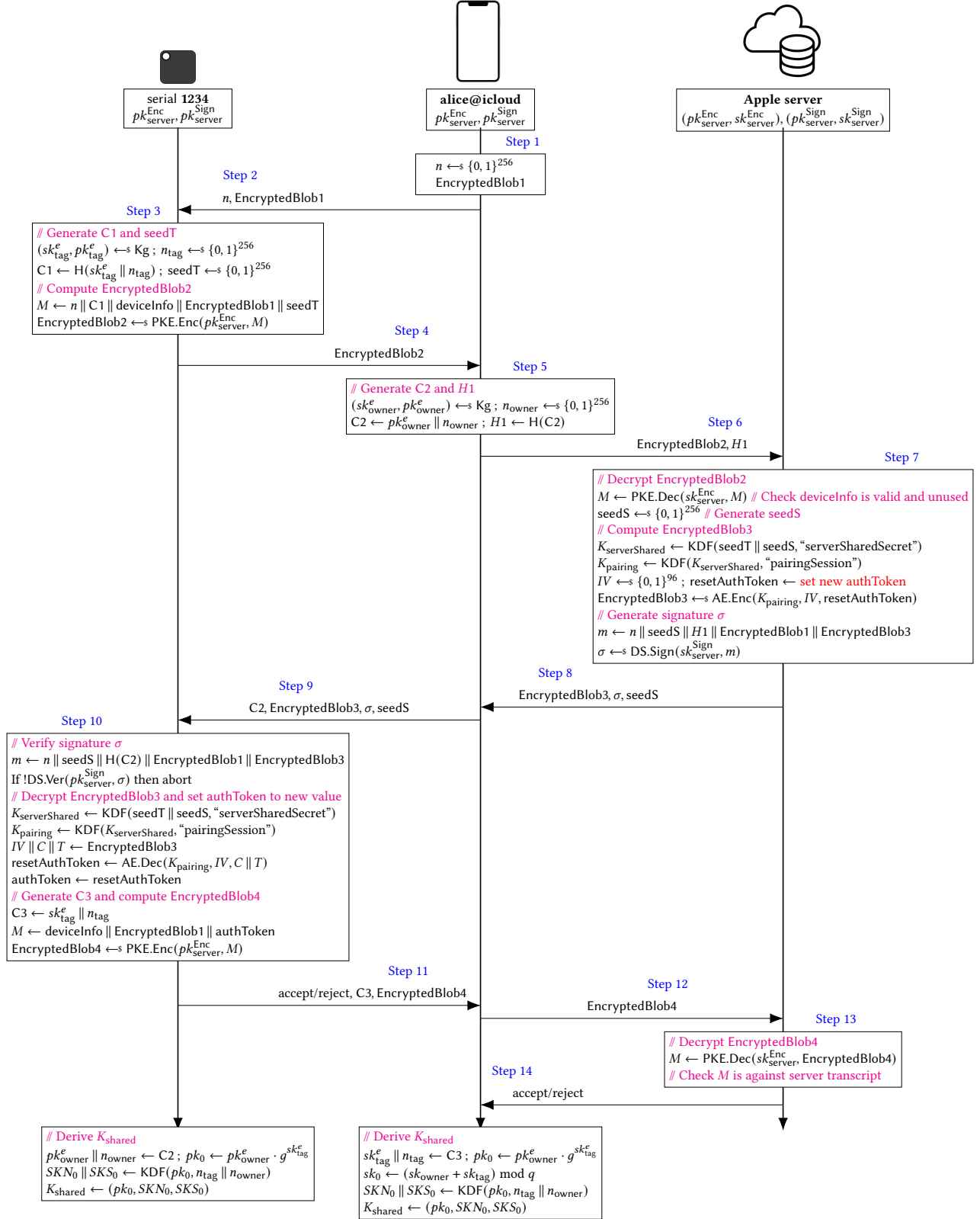


Figure 3: Pairing for Apple's Find My network.

number, software token, and software UUID are as reflected in its internal database, and the tag has not been registered before. Then it parses M to get $\text{deviceInfo} \parallel \text{EncryptedBlob1} \parallel \text{authToken}$, and samples its own 32-byte seed seedS . The server derives its shared secret with the tag $K_{\text{serverShared}}$ by applying a key derivation function KDF to seedT and seedS , and its pairing key K_{pairing} with the tag by applying KDF to $K_{\text{serverShared}}$. After that the server samples a new software token and encrypts it under K_{pairing} and a random IV to get EncryptedBlob3 . Finally, it signs $n \parallel \text{seedS} \parallel H1 \parallel \text{EncryptedBlob1} \parallel \text{EncryptedBlob3}$ to get the signature σ , and σ sends EncryptedBlob3 , σ , seedS to the owner device which in turn forwards them to the tag along with pk_{owner}^e and n_{owner} .

The tag verifies the server's signature σ after reconstructing m locally. It derives $K_{\text{serverShared}}$ and K_{pairing} analogously to the server using seedT and seedS , and uses K_{pairing} to decrypt EncryptedBlob3 and get the new software token. Finally it encrypts the new software token and EncryptedBlob1 under $pk_{\text{server}}^{\text{Enc}}$ to get EncryptedBlob4 and sends EncryptedBlob4 , sk_{tag}^e , and n_{tag} to the owner device. The owner device forwards EncryptedBlob4 to the server which decrypts and verifies it against its transcript. This completes tag authentication.

Shared key derivation. As the last step of the pairing protocol, the tag and the owner establish their shared key. The shared master public key pk_0 is generated as $pk_{\text{owner}}^e \cdot pk_{\text{tag}}^e$ and the secret key sk_0 as $(sk_{\text{owner}}^e + sk_{\text{tag}}^e)$. Both the tag and the owner device also compute SKS_0 and SKN_0 by applying KDF to n_{owner} and n_{tag} . The shared key between the tag and the owner is (pk_0, SKS_0, SKN_0) . Note that based on the information shared, only the owner device can derive sk_0 . This is okay, because ultimately the secret key is used to decrypt location reports which happens on the owner device.

B.2 Pairing for Google's FindHub

We summarize the pairing protocol for Google's FindHub network in Figure 4 and describe it below. For brevity, we group certain steps together and omit those that are not directly relevant to FindHub operations or security. Our description is based on the reverse engineering efforts of Bottger et al. [13], and Google's Find Hub and FastPair specifications [32, 33].

FastPair background. The FindHub pairing protocol builds on Google's *FastPair* protocol [33]. We recall some details of the FastPair protocol that are relevant to the FindHub registration phase. To support FastPair, a tag manufacturer must first register their device model with Google. Once registered, Google assigns the tag a model identifier modelId and an *Anti-Spoofing* public key pair $(sk_{\text{modelId}}, pk_{\text{modelId}})$, which in practice is an Elliptic Curve key pair. The private key sk_{modelId} is embedded in every tag of that model, while Google's servers maintain a mapping from each modelId to its corresponding pk_{modelId} . Notably, all tags sharing the same modelId use the same anti-spoofing key pair.

FindHub registration: Owner preprocessing. The FindHub registration process begins with the server provisioning the owner with a 32-byte owner key K_{owner} .² This key is later used to encrypt other long-term secrets in the FindHub protocol.

²In practice, K_{owner} is sent to the owner encrypted under a key shared with the server and stored in the owner's Google Vault under the "finder_hw" domain.

Then the owner device queries the tag for its modelId . The owner device forwards this modelId to the server which responds with the public anti-spoofing key pk_{modelId} associated with the tag. The owner device then generates its own ephemeral key pair $(sk_{\text{owner}}, pk_{\text{owner}})$ and derives a Diffie-Hellman shared secret K_{DH} as $sk_{\text{owner}} \cdot pk_{\text{modelId}}$. From K_{DH} , the owner device generates a 128-bit key K_{Enc} , by applying the SHA-256 hash function and truncating the output to 128 bits.

Next, the owner device samples three additional keys: (1) a 256-bit AES key K_{AES} , (2) a 128-bit account key K_{account} (the "master" FastPair key, used for mutual authentication), and (3) a 256-bit ephemeral identity key $K_{\text{ephemeralId}}$ (the "master" FindHub key, used to derive future keys and secrets for FindHub operations).

The owner encrypts these keys in a nested fashion using the AES-128-ECB symmetric encryption scheme.

$$\begin{aligned} c_0 &= \text{AES-128-ECB.Enc}(K_{\text{Enc}}, K_{\text{AES}}), \\ c_1 &= \text{AES-128-ECB.Enc}(K_{\text{AES}}, K_{\text{account}}), \\ c_2 &= \text{AES-128-ECB.Enc}(K_{\text{account}}, K_{\text{ephemeralId}}). \end{aligned}$$

Additionally, it also computes the recovery key K_{recovery} by applying the SHA-256 hash function to $K_{\text{ephemeralId}} \parallel 0x01$ and the unwanted tracking protection key $K_{\text{uwTrac Protec}}$ by applying SHA-256 to $K_{\text{ephemeralId}} \parallel 0x03$. The recovery key K_{recovery} is stored on the server to allow the owner to recover $K_{\text{ephemeralId}}$ if it is lost. The purpose of $K_{\text{uwTrac Protec}}$ is explained in Appendix D.3. Finally, the owner sends $(c_0, c_1, c_2, pk_{\text{owner}})$ to the tag.

FindHub registration: Tag preprocessing. The tag uses its private key sk_{modelId} and the received pk_{owner} to derive the shared K_{DH} and compute K_{Enc} as above. It decrypts the ciphertexts in sequence:

$$\begin{aligned} K_{\text{AES}} &= \text{AES-128-ECB.Dec}(K_{\text{Enc}}, c_0), \\ K_{\text{account}} &= \text{AES-128-ECB.Dec}(K_{\text{AES}}, c_1), \\ K_{\text{ephemeralId}} &= \text{AES-128-ECB.Dec}(K_{\text{account}}, c_2). \end{aligned}$$

It then derives K_{recovery} and $K_{\text{uwTrac Protec}}$ analogously to the owner.

To prove knowledge of K_{account} , the tag generates a random 8-byte nonce n and computes the tag h by applying the HMAC-SHA-256 function to $(K_{\text{account}}, 0x01 \parallel n \parallel 0x02 \parallel 0x08)$ and sends (h, n) to the owner.

FindHub registration: Final steps. The owner device authenticates h by comparing it to the locally generated tag h' . If the authentication succeeds, the owner device encrypts K_{account} and $K_{\text{ephemeralId}}$ using AES-GCM encryption scheme under K_{owner} and uniformly random 12-byte nonces n_3 and n_4 to obtain the ciphertexts c_3 and c_4 . Finally, the owner device sends $c_3, c_4, K_{\text{recovery}}$, and $K_{\text{uwTrac Protec}}$ to the server.

This completes registration for the tag. The key $K_{\text{ephemeralId}}$ is the shared secret K_{shared} established between the tag and the owner device and is used for all future FindHub operations.

B.3 Pairing for Samsung's SmartThings Find

In this section, we describe the pairing protocol used in Samsung's SmartThings Find. Our description is based on the reverse engineering efforts of Yu et al. [82] and Liu et al. [53].

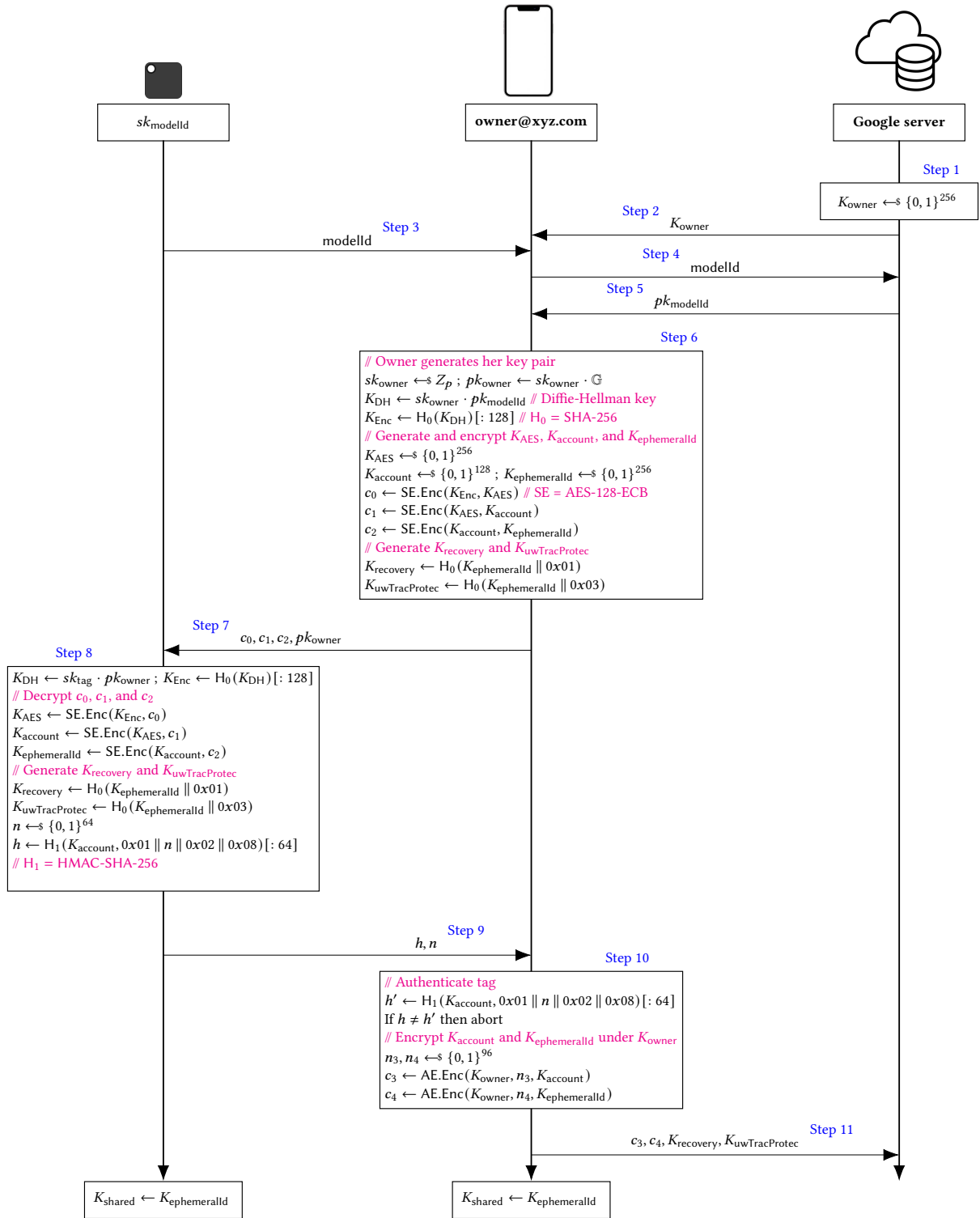


Figure 4: Registration for Google's Find Hub network.

Manufacture-time setup. At manufacture time, each Samsung SmartTag is provisioned a unique 6-byte MAC address $macAddress$.

The long-term secret key of the tag, sk_{tag} is derived deterministically from the $macAddress$ by applying a hash function H_0 . The

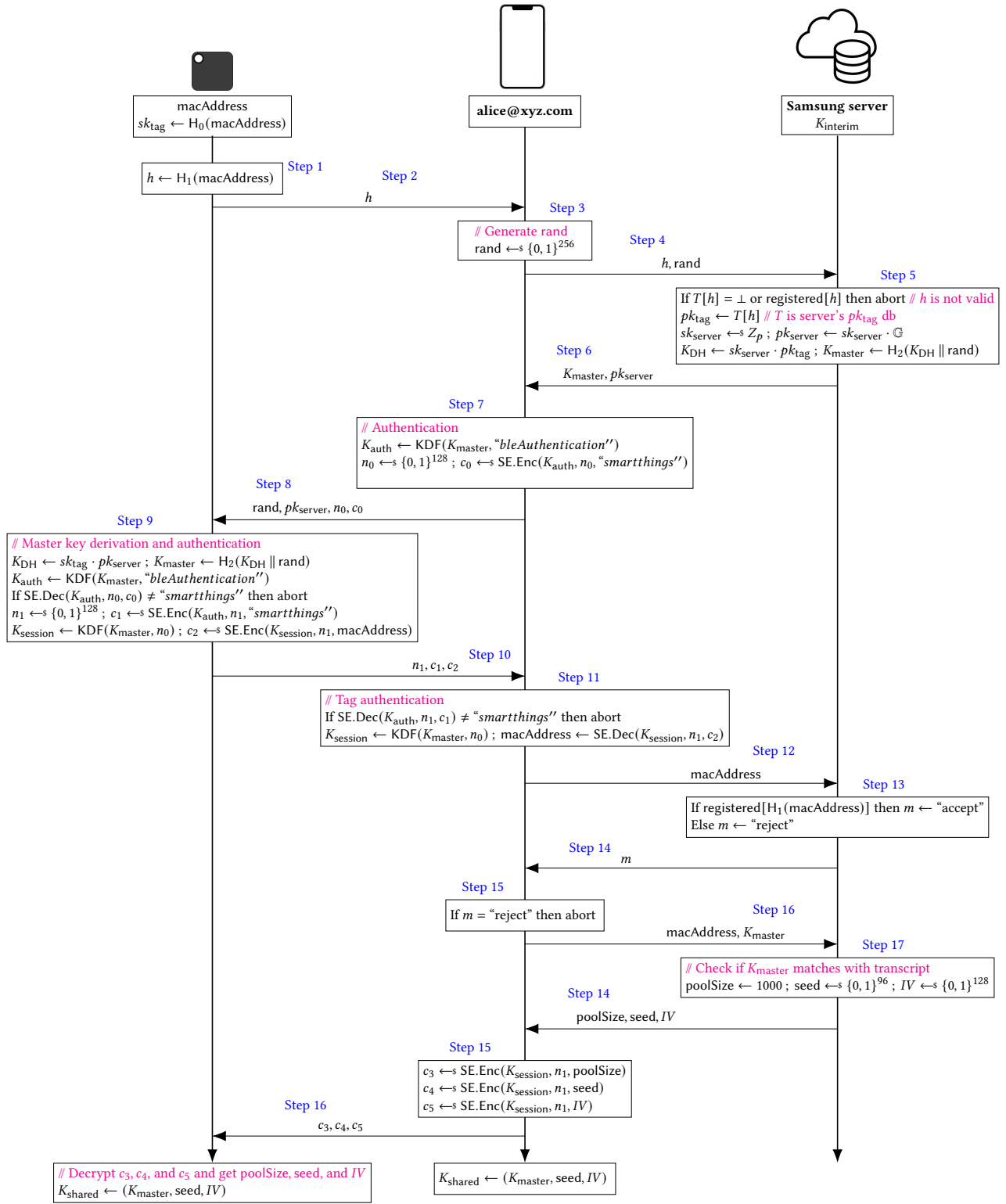


Figure 5: Registration for Samsung's SmartThings Find network.

corresponding public key pk_{tag} is $sk_{\text{tag}} \cdot \mathbb{G}$. The Samsung server retains a database T that maps $H_1(\text{macAddress})$ to pk_{tag} where H_1 is a hash function.

Master key derivation and tag-owner authentication. The pairing process begins when the tag sends $h = H_1(\text{macAddress})$ to the owner device. The owner device samples a 32-byte random value rand and sends h , rand to the server. If the queried MAC address is invalid, i.e., $T[h] = \perp$, the server aborts. Otherwise, the server retrieves pk_{tag} from T , samples its ephemeral key pair $(sk_{\text{server}}, pk_{\text{server}})$, and derives the Diffie-Hellman key $K_{\text{DH}} \leftarrow sk_{\text{server}} \cdot pk_{\text{tag}}$ and the master secret key $K_{\text{master}} \leftarrow H_2(K_{\text{DH}} \parallel \text{rand})$. Then the server sends K_{master} and pk_{server} to the owner device over TLS. The owner device saves K_{master} , derives an authentication key K_{auth} , which is used for mutual authentication between the tag and the owner device, by applying a key derivation function KDF to K_{master} and the string “bleAuthentication.” The owner device then encrypts the string “smarththings” under K_{auth} and a uniformly random 16-byte nonce n_0 to produce the ciphertext c_0 and sends rand , pk_{server} , n_0 , and c_0 to the tag.

The tag derives K_{master} from sk_{tag} and pk_{server} analogously to the server and K_{auth} analogously to the owner device. Using K_{auth} and n_0 , the tag authenticates the owner device by verifying that c_0 decrypts to “smarththings.” Then the tag derives the session key K_{session} by applying KDF to K_{master} and n_0 . The tag subsequently computes two ciphertexts: (1) c_1 by encrypting the string “smarththings” under K_{auth} and a uniformly random string n_1 , and (2) c_2 by encrypting macAddress under K_{session} and n_1 . The tag sends n_1 , c_1 , and c_2 to the owner device. The owner device authenticates the tag by decrypting c_1 using K_{auth} and n_1 , and obtains the macAddress by decrypting c_2 using K_{session} and n_1 .

Pairing finalization. The the owner device sends macAddress to the server, which returns “accept” if the tag wasn’t already registered, and “reject” otherwise. If the server rejected the pairing, the owner device aborts. Otherwise, it sends macAddress , K_{master} to the server which verifies that K_{master} matches its internal records and sets three parameters: (1) a poolSize which is always set to 1000, (2) a uniformly random 12-byte seed, and (3) a 16-byte IV which is a fixed value for all tags. The server sends poolSize , seed, and IV with the owner device which in turn shares it with the tag encrypted under K_{session} and n_1 . The shared key K_{shared} is $(K_{\text{master}}, \text{seed}, IV)$.

B.4 Pairing for Tile’s protocol

We now describe Tile’s pairing protocol. At manufacture time, each tag of a particular model is provisioned the same 16-byte interim key K_{interim} and a unique 8-byte tile_uuid . The server maintains a mapping between tile_uuid s and K_{interim} values.

The tag initiates pairing by sending its tile_uuid to the owner device. The owner device generates a uniformly random 14-byte string randA and sends it to the tag. The tag generates its own 10-byte secret randT , and computes the authentication string sresT as $H(K_{\text{interim}}, \text{randA} \parallel \text{randT} \parallel \text{tile_uuid})$ where H is a hash function. It also derives the shared key K_{shared} as $H(K_{\text{interim}}, \text{sresT})$. Then the tag sends randT and sresT to the owner device which forwards it to the server along with randA and tile_uuid . The server authenticates the tag by verifying sresT and ensuring that the tile_uuid is

not already registered, and derives K_{shared} analogously to the tag. Finally, the server sends K_{shared} to the owner device, completing the pairing phase.

C CONNECTED MODE IMPLEMENTATIONS

C.1 Apple’s FindMy

Once a tag is registered with its owner device, it begins broadcasting BLE advertisements containing a 4-byte ephemeral identifier id which rotates every 15 minutes. The identifiers are generated from K_{shared} as shown in Figure 7. The tag parses K_{shared} to get (pk_0, SKN_0, SKS_0) . Here SKS is the separated mode key and SKN is the connected mode key. For deriving the identifier for the i^{th} epoch id_i , the tag first computes SKN_i by applying a KDF to SKN_{i-1} , then derives $u_i \parallel v_i$ by applying a KDF to SKN_i . Finally, id_i is computed as $pk_0^{u_i} \cdot g^{v_i}$. When broadcasting, the tag uses the first six bytes of id_i as the advertisement address and only includes the first two bites of the first byte in the payload. Though id_i is a public key, it is never broadcast or used as such.

C.2 Google’s FindHub

Once a tag is registered with its owner device, it begins broadcasting BLE advertisements containing an ephemeral identifier id . The identifier can be 20 or 32 bytes long, depending on whether the tag uses Bluetooth 4.0 or Bluetooth 5.0, respectively.

Identifier generation. The connected-mode protocol sets the epoch duration Δ_{conn} to $1024 = 2^{10}$ seconds and defines a rotation exponent $\text{rotExp} = 10$. The tag generates id_i as described in Figure 8. The time counter timeCtr is computed as the time (in seconds) since the tag was activated, with the lowest rotExp bits set to 0. The value r' is obtained by encrypting the value $(0xFF \parallel \text{rotExp} \parallel \text{timeCtr} \parallel 0x00 \parallel \text{rotExp} \parallel \text{timeCtr})$ using AES-ECB under $K_{\text{ephemeralId}}$. It is then mapped to the finite field of either SECP160R1 (for 20-byte id_i) or SECP256R1 (for 32-byte id_i) by reducing r' modulo the curve order n to obtain r . Finally, id_i is set to the x -coordinate of the curve point $R = r \cdot \mathbb{G}$. Note that the value of id_i changes with timeCtr which increments every Δ_{conn} seconds.³

Uploading identifiers. The owner device periodically uploads a batch of precomputed future identifiers to the server. When a finder later submits a location report for a lost tag, the report contains the observed identifier. The server uses this identifier to determine which tag it belongs to and associate the report with the correct user account.

The UT byte. Each advertisement includes a one-byte unwanted tracking mode value, UT, that encodes the tag’s operational mode: connected (0x40) or lost (0x41). This value is intended to mitigate unwanted tracking by indicating when a tag is within range of its owner. Tags in connected mode rotate their randomized MAC addresses in sync with id_i . In lost mode, however, MAC addresses are rotated only once every 24 hours.

³To avoid timing-based attacks against unlinkability, Google recommends tag manufacturers to implement small random delays i.e. instead of incrementing timeCtr exactly every Δ_{conn} seconds, it is incremented every $\Delta_{\text{conn}} + \epsilon$ seconds where $\epsilon \leftarrow \$ [1, 204]$ seconds.

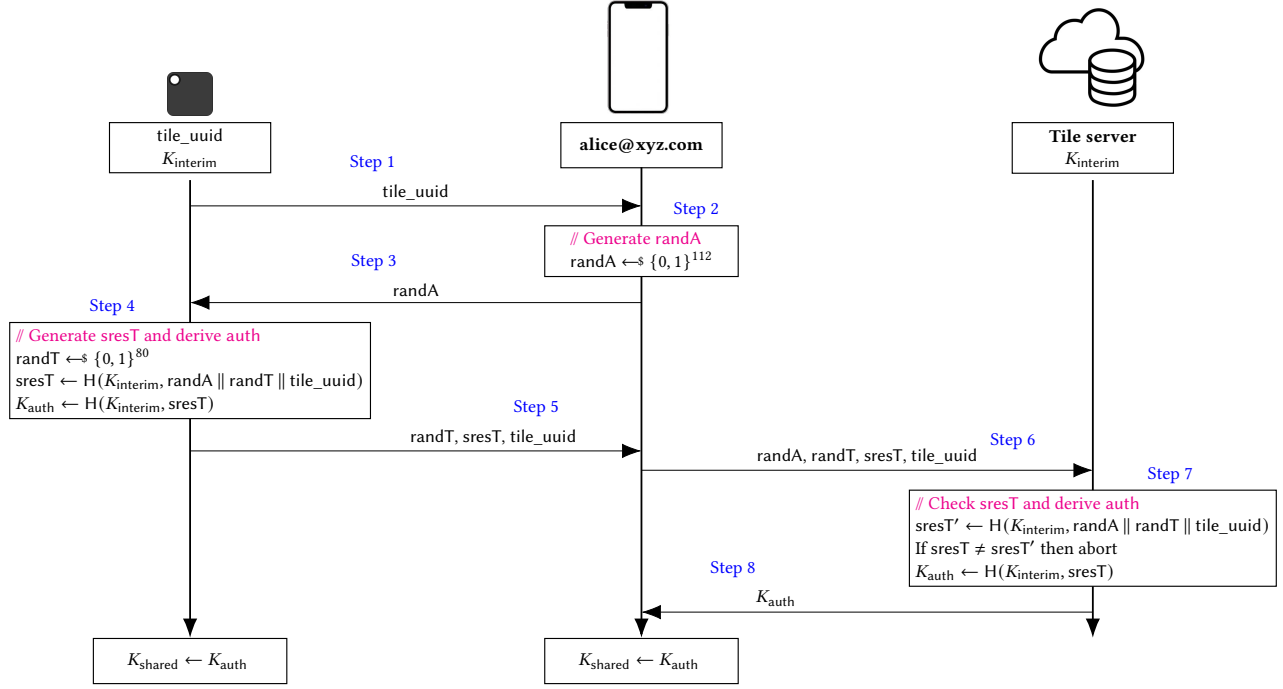


Figure 6: Registration for Tile's protocol.

```

(pk0, SKN0, SKS0) ← Kshared
SKNi ← KDF(SKNi-1, "update")
ui || vi ← KDF(SKNi, "diversify") // |ui| = |vi| = 288, KDF = ANSI-X9.63-KDF
ui ← ui (mod q - 1) + 1
vi ← vi (mod q - 1) + 1
idi ← pk0ui · gvi
  
```

Figure 7: Derivation of connected-mode identifiers for Apple's FindMy protocol.

```

KephemeralId ← Kshared
r'i ← KDF(KephemeralId, 0xFF || rotExp || timeCtr || 0x00 || rotExp || timeCtr) // KDF = AES-256-ECB.Enc
ri ← r'i mod n
idi ← ri · G
  
```

Figure 8: Derivation of connected-mode identifiers for Google's FindHub protocol.

Location reporting. Unlike Apple, Google owner devices periodically submit location reports for their own tags even in connected mode. This is because Google offers an additional feature through its OF protocol that allows an owner to retrieve the last location where they were connected to their tag. These reports are encrypted under $\text{SHA-256}(K_{\text{ephemeralId}})$ using AES-GCM with a fresh 12-byte nonce n .

C.3 Samsung's SmartThingsFind

Once a tag is registered with its owner device, it begins broadcasting BLE advertisements containing an ephemeral 8-byte identifier.

Identifier generation. The connected-mode protocol sets the epoch duration Δ_{conn} to 15 minutes. The tag generates id_i as described in Figure 9. First, the tag computes a privacy ID key K_{privId} by applying a key derivation function KDF to the master key K_{master} derived during pairing. Then it generates poolSize number of unique identifiers by applying the KDF_1 key derivation function to IV and the epoch counter x_i , truncating the output to 8 bytes. Here, IV is the value set by the server during pairing, and x_i is derived deterministically from i .

Recall that the server possesses all the inputs required to generate a tag's identifiers. The server generates these identifiers analogously

```

 $(K_{\text{master}}, \text{seed}, IV) \leftarrow K_{\text{shared}}$ 
 $K_{\text{privId}} \leftarrow \text{KDF}_0(K_{\text{master}}, \text{"privacy"}) \parallel \text{KDF}_0 = \text{SHA-256}$ 
For  $i = 1$  to  $\text{poolSize}$  do  $x_i \leftarrow 0 \parallel (i \wedge 255) \parallel \text{seed} \parallel 0 \parallel (i \wedge 255)$ 
 $id_i \leftarrow \text{KDF}_1(K_{\text{privId}}, IV, x_i)[1:64] \parallel \text{KDF}_1 = \text{AES-CBC-256}$ 

```

Figure 9: Derivation of connected-mode identifiers for Samsung’s protocol.

to the tag and maintains a map of identifiers to the corresponding owner. This map will become relevant during the lost mode interactions which we describe in Appendix D.

Advertisement data. In addition to the 8-byte id_i , the tag includes a status byte to encode the state of the tag, a 3-byte time counter timeCtr which increments every 15 minutes, a 4-byte authentication tag which is produced by MAC-ing the status byte, id_i , and timeCtr and truncating the output. Additionally, the advertisement payload contains an E2E field which is set to 0 by default and 1 when the owner enables end-to-end encryption. We describe this optional feature in Appendix D. After every 15-minute interval, a tag in the connected mode updates id_i , timeCtr , the authentication tag, and the broadcast MAC address.

C.4 Tile’s protocol

Tile’s protocol does not differentiate between lost and connected modes; tags broadcast similarly in both modes, rotating identifiers every 15 minutes. We show Tile’s identifier generation process in Figure 10. After registration, tags generate 8640 identifiers from K_{shared} . First, a seed key K_{seed} is generated by applying a KDF to K_{shared} . Then, the i^{th} identifier id_i for $i \in [1, 8640]$ is generated by applying KDF to K_{seed} and i and truncating the output to 8 bytes. Once a tag has rotated through 8640 identifiers, it cycles back through the same values.

D LOST MODE IMPLEMENTATIONS

D.1 Apple’s FindMy

We now discuss Apple’s implementation of lost mode interactions in its Find My network. The implementation for generating lost-mode identifiers is the same as the one for generating connected-mode identifiers, except the separated mode key SKS is used instead of the near mode key SKN . Moreover, the identifiers are rotated on in 24 hours instead of every 15 minutes. Additionally, tags encode their state in the advertisement payload. Bit 2 of byte 2 of the advertisement payload indicates the mode of tag. It is set when connected to an owner device (indicating near mode) and unset after being disconnected for 15 minutes (indicating lost mode). Nearby finder devices only report locations for tags in the lost mode. Similarly to the near mode, lost mode advertisements use the first six bytes of the lost mode identifier as the advertising address. However, the additionally include the remaining bytes of the identifier in the advertisement payload. In the lost mode, the identifier is a public key and is used by nearby finders to encrypt location reports.

D.2 Samsung’s SmartThings Find

We now describe Samsung’s implementation of the lost mode in SmartThings Find.

Mode transitions. A SmartTag enters *offline mode* if it has been disconnected from its owner for 15 minutes, and *overmature offline mode* if it remains offline for 24 hours. The tag’s mode is encoded in bits 5–7 of Byte 0 of the advertisement payload (010 for offline mode and 011 for overmature offline mode). Lost-mode identifiers are generated using the same procedure as connected-mode identifiers. Finder devices only report locations for tags that are in either offline or overmature offline mode.

Optional end-to-end encryption. By default, finder devices upload location reports to the server in plaintext over TLS. The server associates each report with the correct owner by using the identifier mapping described in Appendix C.3. When an owner later queries the locations of its tags, the server returns all reports corresponding to that owner.

Samsung, however, provides an optional end-to-end encryption (E2E) mode that users can explicitly enable. Tags with E2E enabled indicate this setting via a dedicated field in their advertisements. When a user enables the E2E mode, the user is first required to choose a 6-digit PIN. Then the owner device generates a key pair (pk_e, sk_e) , encrypts sk_e using the hash of the PIN and a random IV , producing ciphertext c , and sends (pk_e, c, IV) to the server. Finders that encounter such tags request pk_e from the server by querying the recorded identifier, encrypt the location under pk_e , and send the ciphertext together with the tag’s identifier back to the server. Finally, the owner decrypts the reports locally using sk_e .

D.3 Google’s Find My Device Network

We now discuss Google’s implementation of lost mode interactions in its Find My Device Network. Google’s implementation for generating lost-mode identifiers is the same as the one for generating connected-mode identifiers. However, in the lost mode, tags in Google’s FMDN enter the Unwanted Tracking (UT) mode where they slow down the rate of MAC addresses randomization to once in 24 hours.

End-to-end encryption of location data. Recall that the identifiers broadcast by a tag are ephemeral public keys. By default, a FMDN finder encrypts location reports using the observed identifier, discarding any reports for locations within 100m of their home location. Each submission to the server includes the encrypted location report, the (truncated) identifier of the tag, and the UT status observed in the advertisement.

Mode transitions. In Apple’s protocol, a tag automatically switches between connected and lost modes based on Bluetooth proximity

$$\begin{aligned} K_{\text{seed}} &\leftarrow \text{KDF}(K_{\text{shared}}, \text{tile_uuid} \parallel \text{"identity"}) \text{ // KDF = HMAC-SHA-256} \\ id_i &\leftarrow \text{KDF}(K_{\text{seed}}, i)[64] \end{aligned}$$

Figure 10: Derivation of connected-mode identifiers for Tile's protocol.

$$\begin{aligned} (pk_0, SKN_0, SKS_0) &\leftarrow K_{\text{shared}} \\ SKS_i &\leftarrow \text{KDF}(SKS_{i-1}, \text{"update"}) \\ u_i \parallel v_i &\leftarrow \text{KDF}(SKS_i, \text{"diversify"}) \text{ // } |u_i| = |v_i| = 288, \text{ KDF = ANSI-X9.63-KDF} \\ u_i &\leftarrow u_i \pmod{q-1} + 1 \\ v_i &\leftarrow v_i \pmod{q-1} + 1 \\ id_i &\leftarrow u_i \cdot pk_0 + v_i \cdot \mathbb{G} \end{aligned}$$

Figure 11: Derivation of lost-mode identifiers for Apple's FindMy protocol. Identifiers are rotated every $\Delta_{\text{lost}} = 24$ hours.

to its owner device. In contrast, Google's FMDN relies on the server to control this transition. When a finder device submits a location report for the tag, the server checks the included UT status. If UT mode is not enabled, the server extracts the truncated identifier from the report and checks the last time the corresponding owner submitted a location report for it. If more than 30 minutes have passed since the owner of the tag reported its location, the server responds to the finder with an HTTPS message containing the unwanted tracking protection key $K_{\text{uwTracProtec}}$ (see Appendix B.2) instructing it to set the UT byte of the tag. The UT bit can only be cleared when the tag reconnects to its owner device.